



Hausarbeit

Beispieltitel
für einen Bericht

Autor: Vorname Nachname
Matrikel-Nr.: xxxxxxxx
Studiengang: Maschinenbau

Erstprüfer: Prof. Dr. Elmar Wings
Abgabedatum: 26. Mai 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	5
Liste des Listings	7
1. Introduction	11
I. Domain Knowledge - Application	13
2. Applikationen	15
2.1. Mikrocontroller-Programmierung mit der Arduino IDE	15
2.1.1. Allgemeines zur Arduino IDE	15
2.1.2. Das Steuerungsprogramm des Ultraschallradars	15
2.1.3. Quellcode der Arduino-Steuerung	16
2.2. Benutzeroberfläche und Visualisierung mit Processing	18
2.2.1. Allgemeines zu Processing	18
2.2.2. Das Ultraschallradar-Programm	18
2.2.3. Quellcode der Processing-Anwendung	19
II. Domain Knowledge - Hardware	23
3. Arduino Nano 33 BLE Sense Lite	25
3.1. Arduino Nano 33 BLE Sense	25
3.1.1. Arduino Nano 33 BLE Sense Lite	26
4. Arduino Tiny Machine Learning Kit	27
4.1. Arduino Nano 33 BLE Sense Lite	27
4.1.1. Pinbelegung	27
4.1.2. Mechanik	28
4.1.3. Elektronik	29
4.1.4. Verbaute Sensoren	29
4.2. On-Board Sensor Description	30
4.2.1. Gesture, Proximity, and Color Detection Sensor ADPS-9960 . .	32
4.2.2. Accelerometer, Gyroscope, and Magnetometre Sensor LSM9DS1	32
4.2.3. Pressure Sensor LPS22HB	34
4.2.4. Relative Humidity and Temperature Sensor HTS221	34
4.2.5. Digital Microphone MP34DT05-A	35
4.2.6. Bluetooth Module nRF52840	35
4.3. Bluetooth Module INA B306	37
4.4. Arduino Nano 33 BLE Pin Configuration	37
4.5. Was fehlt	37
4.6. Further Readings	38
5. Ultraschallsensor HC-SR04	39
5.1. Allgemeines	39

5.2.	Ultraschallsensor HC-SR04	39
5.3.	Spezifikation	39
5.4.	Bibliothek	40
5.4.1.	Beschreibung	40
5.4.2.	Installation	40
5.4.3.	Funktionen	40
5.5.	Beispiel	42
5.5.1.	Handbuch	42
5.5.2.	Funktion vom Beispiel	43
5.5.3.	Code	43
5.6.	Kalibrierung	44
5.7.	Einfacher Code ohne newPing Bibliothek	44
5.8.	Einfache Anwendung	45
5.9.	Tests	46
5.9.1.	Einfacher Funktionstest	46
5.9.2.	Alle Funktionen testen	46
5.10.	Weiterführende Literatur	47
6.	SG90 Servomotor	49
6.1.	Allgemeines	49
6.2.	Servomotor TowerPro SG90	49
6.3.	Spezifikation	49
6.4.	Bibliothek	50
6.4.1.	Beschreibung	50
6.4.2.	Installation	50
6.4.3.	Funktionen	51
6.5.	Beispiel	51
6.5.1.	Handbuch	51
6.5.2.	An einem Beispiel	51
6.5.3.	Code	51
6.5.4.	Dateien	52
6.6.	Kalibrierung	52
6.7.	Einfacher Code (ohne Bibliothek)	53
6.8.	Einfache Anwendung	54
6.9.	Tests	54
6.9.1.	Einfacher Funktionstest	54
6.9.2.	Alle Funktionen testen	54
6.10.	Weiterführende Literatur	54
III.	Domain Knowledge - Tools	55
7.	Python Tool XY	57
8.	Hardware Tool XY	59
IV.	Developmenmt	61
9.	Flow Chart Development XY	63
V.	Monitoring	65
10.	Monitoring XY	67

VI. Verification - Evaluation - Conclusion	69
11. Evaluation/Verification XY	71
12. To DoX Y	73
13. Conclusion XY	75
 VII. Anhang	 77
14. Bill Of Material	79
15. SBOM	81
16. Package Example	83
16.1. Introduction	83
16.2. Description	83
16.3. Installation	83
16.4. Example - Manual	83
16.5. Example	83
16.6. Example - Code	83
16.7. Example - Files	83
16.8. Further Reading	83
 VIII. Hints - Examples for LaTeX - Read, Use and Remove	 85
17. Hinweise	87
17.1. Allgemeine mathematische Beschreibung Bézier-Kurve	88
18. Verrundung mit einem Kreisbogen	91
18.1. Gleichungen	91
18.2. Bewertung	93
18.3. Beispiele	95
19. Maple-Dateien	99
19.1. Geometrien mit Maple	99
19.2. Verwendete Geometrieelemente	99
19.3. Aufbau der Datenstruktur	100
19.4. Struktur eines Moduls	100
19.4.1. Genereller Aufbau eines Moduls	102
19.4.2. Sicherung eines Moduls	104
19.4.3. Verwendung eines Moduls	104
19.4.4. Erstellung eines Maple-Moduls	104
19.5. Funktionen der Module	105
19.5.1. Modul MPoint	105
19.5.2. Modul MLine	106
19.5.3. Modul MArc	107
19.5.4. Modul MBezier	107
19.5.5. Modul MPolygon	108
19.5.6. Modul MGeoList	108
19.5.7. Modul MHermiteProblem	109
19.5.8. Modul MHermiteProblemSym	109
19.5.9. Modul MConstant	110
19.5.10. Modul MGeneralMath	110

19.5.11Modul Biarc	111
19.5.12Modul MBiarc	111
19.6. Programmablaufplan	112
19.6.1. Gesamauf	112
19.6.2. Verrundung der Kurve	112
20.Representation of Python Programs	115
21.Representation of Programs written for Arduino Boards	117
22.Erstes Kapitel	119
23.CAGD	121
A. Zeichnungen mit tikz	123
B. Kriterien für eine gutes $\text{\LaTeX}$-Projekt	125
Index	127

Abbildungsverzeichnis

3.1. Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store	26
4.1. Arduino Nano 33 BLE Sense Architektur	29
4.2.	31
4.3. Pin assignment of the Arduino Nano 33 BLE Sense	32
5.1. Schematische Darstellung des Ultraschallsensors HC-SR04	40
5.2. Schaltplan Ultraschallsensor HC-SR04	41
6.1. Schematische Darstellung des Servomotors TowerPro SG90	50
17.1. Bernstein-Polynom vom Grad 3	89
17.2. Bézier-Kurve zum Beispiel 17.1	90
18.1. Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck	91
18.2. Winkeländerung bei einer Verrundung mit einem Kreisbogen	93
18.3. Krümmungsverlauf bei der Verrundung mit einem Kreisbogen	94
18.4. Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$	94
18.5. Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen	94
19.1. Programmablaufplan „Eckenverrundung“	113
19.2. Programmablaufplan „Auswahl der Verrundungsstrategien“	114

Liste des Listings

4.1. Simple example using of the builtin microphone of the Arduino Nano	
33 BLE Sense	36
5.1. Beispiel Code	43
5.2. Beispiel Code ohne Bibliothekseinbindung	44
5.3. Einparkhilfe Beispielcode	45
6.1. Beispiel Code für den TowerPro SG90	51
6.2. Code ohne Bibilothek für den TowerPro SG90	53
20.1. „Hello World“ in Python – Variant 1	116
21.1. „Hello World“ in Python – Variant 1	118

Abkürzungen

AOP Acoustic Overload Point

CNC Computerized Numerical Control

IMU Inertialmesseinheit

IoT Internet of Things

PDM Pulse Density Modulation

SPS Speicherprogrammierbare Steuerung

1. Introduction

Flettner rotors are now widely used [1,2,3]. ...

The construction of a Flettner rotor for a water taxi [4] ...

External influences pose great challenges to [5,6] ...

Through the use of 3D printed parts, ...

In the second chapter, ...

Teil I.

Domain Knowledge - Application

2. Applikationen

2.1. Mikrocontroller-Programmierung mit der Arduino IDE

2.1.1. Allgemeines zur Arduino IDE

Die Arduino Integrated Development Environment (IDE) ist eine plattformunabhängige Open-Source-Entwicklungsumgebung, die speziell für die Programmierung von Mikrocontrollern entworfen wurde. Sie basiert im Kern auf der Programmiersprache C/C++, verbirgt jedoch viele komplexe hardwarenahe Konfigurationen hinter einer stark vereinfachten Syntax und einer umfangreichen Standardbibliothek.

Ein klassisches Arduino-Programm (auch „Sketch“ genannt) ist strukturell immer in zwei wesentliche Hauptfunktionen unterteilt: Die `setup()`-Funktion, welche genau einmal beim Systemstart ausgeführt wird und für die Initialisierung von Pins und Schnittstellen zuständig ist, sowie die `loop()`-Funktion, welche im Anschluss als Endlosschleife die eigentliche Arbeitslogik des Mikrocontrollers abarbeitet. Die fertigen Programme werden direkt aus der IDE heraus kompiliert und über eine serielle USB-Verbindung auf den Flash-Speicher des Mikrocontrollers übertragen.

2.1.2. Das Steuerungsprogramm des Ultraschallradars

Der Quellcode für den Arduino Nano 33 BLE übernimmt die direkte Hardwaresteuerung der Sensorik, der Aktuatorik sowie der visuellen Statusanzeigen (LEDs). Bevor das System in den regulären Betriebsmodus wechselt, durchläuft es in der `setup()`-Phase einen sogenannten Power-On Self-Test (POST). Hierbei teilt der Mikrocontroller der verbundenen Processing-Software zunächst den Status `S:TESTING` mit. Daraufhin werden die LEDs auf Funktion geprüft, der Servomotor absolviert eine Testdrehung und der Ultraschallsensor führt eine initiale Referenzmessung durch.

Nur wenn diese Testmessung valide Werte (zwischen 1 cm und 400 cm) liefert, wird das System freigegeben, die grüne Status-LED aktiviert und das Signal `S:OK` an den Computer gesendet. Tritt ein Fehler auf, beispielsweise durch einen Verbindungsabbruch zum Sensor (Timeout), leuchtet die rote LED auf, das System blockiert aus Sicherheitsgründen den weiteren Betrieb und meldet `S:ERR_SENSOR`.

Im laufenden und fehlerfreien Betrieb (`loop`) iteriert das System kontinuierlich durch die Positionen des Servomotors (0° bis 180° und zurück). Nach jedem Positionsschritt pausiert das System kurz, um die mechanische Trägheit des Motors auszugleichen. Anschließend feuert der HC-SR04-Sensor einen 10 µs langen Ultraschall-Impuls ab und misst die Zeit bis zum Eintreffen des Echos. Aus dieser Laufzeit errechnet der Mikrocontroller unter Einbeziehung der Schallgeschwindigkeit die genaue Distanz.

Abschließend formatiert der Arduino diese Wertepaare in das speziell für dieses Projekt definierte Kommunikationsprotokoll (`D:Winkel,Distanz`) und übermittelt sie in Echtzeit an die serielle Schnittstelle zur weiteren grafischen Auswertung.

2.1.3. Quellcode der Arduino-Steuerung

Listing 2.1: Arduino Steuerungscode mit Doxygen-Dokumentation

```

1  #include <Servo.h>
2
3  /**
4   * @file RadarControl.ino
5   * @brief Steuerungscode fuer das Ultraschallradar mit Arduino Nano 33 BLE.
6   * * Dieser Code steuert einen SG90 Servomotor und liest Distanzen ueber einen
7   * HC-SR04 Ultraschallsensor aus. Die Daten werden ueber die serielle
8   * Schnittstelle an ein Processing-Frontend gesendet. Zudem geben zwei LEDs
9   * (Rot/Gruen) visuelles Feedback ueber den Systemstatus.
10  */
11
12  // --- PIN DEFINITIONEN ---
13  /** @brief Pin fuer den Trigger-Anschluss des Ultraschallsensors. */
14  const int trigPin = 4;
15  /** @brief Pin fuer den Echo-Anschluss des Ultraschallsensors (Spannungsteiler beachten!). */
16  const int echoPin = 5;
17  /** @brief PWM-Pin zur Steuerung des Servomotors. */
18  const int servoPin = 9;
19  /** @brief Pin fuer die gruene Status-LED (System OK). */
20  const int ledGreen = 2;
21  /** @brief Pin fuer die rote Status-LED (Systemfehler). */
22  const int ledRed = 3;
23
24  /** @brief Servo-Objekt zur Steuerung des Motors. */
25  Servo radarServo;
26
27  // --- SYSTEM VARIABLEN ---
28  /** @brief Gibt an, ob der Hardware-Selbsttest erfolgreich war. */
29  bool systemOk = false;
30  /** @brief Speichert den aktuellen Winkel des Servomotors (0 bis 180 Grad). */
31  int currentAngle = 0;
32  /** @brief Bestimmt die Drehrichtung des Servos (1 fuer Vorwaerts, -1 fuer Rueckwaerts). */
33  int direction = 1;
34  /** @brief Maximale Messdistanz in cm (wichtig fuer die Skalierung im Radar). */
35  const int maxDistance = 40;
36
37  /**
38   * @brief Initialisiert die Pins, startet die serielle Kommunikation und fuehrt einen Selbsttest durch.
39   * * Waehrend des Selbsttests (POST - Power-On Self-Test) werden die LEDs kurz aufgeleuchtet,
40   * der Servo macht eine Testdrehung und der Sensor macht eine Probemessung.
41   * Das Ergebnis ("S:OK" oder "S:ERR_SENSOR") wird an Processing gesendet.
42   */
43  void setup() {
44      // Pins konfigurieren
45      pinMode(trigPin, OUTPUT);
46      pinMode(echoPin, INPUT);
47      pinMode(ledGreen, OUTPUT);
48      pinMode(ledRed, OUTPUT);
49
50      radarServo.attach(servoPin);
51      radarServo.write(0); // Servo auf Startposition
52
53      Serial.begin(9600);
54
55      // WICHTIG FUER NANO 33 BLE: Warten, bis PC zuhoert
56      while (!Serial);
57
58      // --- START DES SELBSTTESTS (POST) ---
59      Serial.println("S:TESTING");
60
61      // 1. LED Test
62      digitalWrite(ledGreen, HIGH);
63      digitalWrite(ledRed, HIGH);
64      delay(1000);

```

```
65 digitalWrite(ledGreen, LOW);
66 digitalWrite(ledRed, LOW);
67
68 // 2. Servo Test (Einmal hin und her)
69 for(int i = 0; i <= 180; i+=30) {
70     radarServo.write(i);
71     delay(100);
72 }
73 radarServo.write(0);
74 delay(500);
75
76 // 3. Sensor Test
77 long testDist = getDistance();
78
79 // Wenn Distanz 0 ist (Timeout) oder voellig unlogisch, Fehler melden
80 if (testDist <= 0 || testDist > 400) {
81     systemOk = false;
82     Serial.println("S:ERR_SENSOR");
83     digitalWrite(ledRed, HIGH); // Rote LED dauerhaft AN
84 } else {
85     systemOk = true;
86     Serial.println("S:OK");
87     digitalWrite(ledGreen, HIGH); // Gruene LED dauerhaft AN
88 }
89 }
90
91 /**
92  * @brief Hauptschleife. Fuehrt den Radar-Scan durch, falls das System fehlerfrei ist.
93  * * Bewegt den Servo schrittweise, misst die Distanz und sendet die Daten im Format
94  * * "D:Winkel,Distanz" an die serielle Schnittstelle. Prallt der Servo an den
95  * * Grenzen (0 oder 180 Grad) ab, wird die Richtung umgekehrt.
96  */
97 void loop() {
98     if (systemOk) {
99         // Radar bewegen
100         radarServo.write(currentAngle);
101         delay(30); // Kurze Pause, damit der Servo die Position erreicht
102
103         // Distanz messen
104         int distance = getDistance();
105
106         // Daten an Processing senden (Format: D:Winkel,Distanz)
107         Serial.print("D:");
108         Serial.print(currentAngle);
109         Serial.print(",");
110         Serial.println(distance);
111
112         // Winkel fuer den naechsten Durchlauf anpassen
113         currentAngle += direction;
114         if (currentAngle >= 180) {
115             direction = -1;
116         } else if (currentAngle <= 0) {
117             direction = 1;
118         }
119     } else {
120         // Im Fehlerfall passiert hier nichts weiter, ausser dass die rote LED leuchtet.
121         // Das System muss neugestartet werden (Reset-Button).
122         delay(1000);
123     }
124 }
125
126 /**
127  * @brief Misst die Distanz mithilfe des Ultraschallsensors.
128  * * Sendet einen 10 Mikrosekunden langen Trigger-Impuls und misst die Zeit
129  * * bis zum Eintreffen des Echos ueber pulseIn().
130  * * @return Gemessene Distanz in Zentimetern (cm) oder 0 bei einem Timeout.
131  */
132 long getDistance() {
133     digitalWrite(trigPin, LOW);
```

```

134   delayMicroseconds(2);
135   digitalWrite(trigPin, HIGH);
136   delayMicroseconds(10);
137   digitalWrite(trigPin, LOW);
138
139   // Timeout von 30.000 Mikrosekunden (ca. 5 Meter), verhindert Aufhaengen
140   long duration = pulseIn(echoPin, HIGH, 30000);
141
142   if (duration == 0) return 0; // Timeout = Fehler
143
144   long dist = duration * 0.034 / 2; // Umrechnung in cm
145   return dist;
146 }

```

2.2. Benutzeroberfläche und Visualisierung mit Processing

2.2.1. Allgemeines zu Processing

Processing ist eine Open-Source-Programmiersprache und integrierte Entwicklungsumgebung (IDE), die ursprünglich für die Bereiche visuelle Kunst, Design und elektronische Medien entwickelt wurde. Sie basiert auf der Programmiersprache Java, bietet jedoch eine stark vereinfachte Syntax und eine speziell auf grafische Anwendungen zugeschnittene Bibliothek. Dadurch ermöglicht Processing es, komplexe visuelle Ausgaben, Animationen und Interaktionen mit nur wenigen Zeilen Code zu realisieren. Ein zentraler Vorteil von Processing im Kontext von Mikrocontroller-Projekten ist die native und unkomplizierte Unterstützung der seriellen Kommunikation. Dies macht es zu einem idealen grafischen Frontend für Arduino-Anwendungen. Während der Mikrocontroller die zeitkritische Hardware-Steuerung und Sensordaten-Erfassung übernimmt, kann Processing diese Rohdaten am Computer in Echtzeit auswerten und grafisch aufbereiten.

2.2.2. Das Ultraschallradar-Programm

Für das vorliegende Projekt wird Processing genutzt, um die vom Arduino Nano 33 BLE gesendeten Sensordaten (Winkel und Distanz) abzufangen und in eine klassische Radar-Anzeige zu übersetzen. Die Software arbeitet dabei als Zustandsmaschine (State Machine) mit den Zuständen **TESTING**, **OK** und **ERROR**. Während der Arduino beim Systemstart kalibriert wird, zeigt die Benutzeroberfläche einen Ladebildschirm an. Erst wenn das Signal **S:OK** über die serielle Schnittstelle empfangen wird, wechselt die Software in den Live-Scan-Modus.

Die Visualisierung simuliert das Nachleuchten eines echten Radars durch das wiederholte Zeichnen eines teiltransparenten Hintergrunds. Das Interface besteht aus einem Koordinatensystem mit konzentrischen Ringen, welche die Entfernungen (10 cm bis 40 cm) repräsentieren, sowie Hilfslinien für die verschiedenen Winkel. Die aktuelle Ausrichtung des Servomotors wird als grüne Scan-Linie dargestellt.

Sobald ein Objekt innerhalb der definierten Reichweite von 40 cm erkannt wird, transformiert das Programm die Polarkoordinaten (Winkel und Distanz) in kartesische Koordinaten (X/Y-Pixelwerte). Das Hindernis wird daraufhin als roter Punkt im Raster eingezeichnet. Zusätzlich generiert die Software eine auffällige, blinkende Warnmeldung („OBJEKT ERKANNT!“) sowie eine dynamische Textausgabe, um den Nutzer visuell auf das Hindernis hinzuweisen.

Um die genaue Funktionsweise und Struktur der Software nachvollziehbar zu machen, ist der Doxygen-dokumentierte Quellcode im Folgenden eingeblendet.

2.2.3. Quellcode der Processing-Anwendung

Listing 2.2: Processing Radar-Visualisierung

```

1  import processing.serial.*;
2
3  /**
4   * @file RadarVisualisierung.pde
5   * @brief Empfängt serielle Daten vom Arduino und visualisiert diese als Radar.
6   * * Diese Software dient als grafisches Frontend fuer ein Ultraschallradar.
7   * Sie kommuniziert ueber den seriellen Port und stellt Hindernisse in
8   * Echtzeit auf einem simulierten Radar-Display dar.
9   */
10
11  Serial myPort;
12  String systemState = "TESTING";
13  String errorMessage = "";
14
15  float angle = 0;
16  float distance = 0;
17
18  // Einstellungen fuer das Radar
19  int radarRadius = 400;
20  int maxDistanceCm = 40;
21
22  /**
23   * @brief Initialisiert das Fenster und die serielle Kommunikation.
24   */
25  void setup() {
26    size(1000, 600);
27    smooth();
28
29    // COM-PORT EINRICHTUNG
30    try {
31      // Hier den direkten Pfad zum Mac-USB-Port eintragen
32      String portName = "/dev/cu.usbmodem1401";
33
34      myPort = new Serial(this, portName, 9600);
35      myPort.bufferUntil('\n');
36    } catch (Exception e) {
37      systemState = "ERROR";
38      errorMessage = "Konnte COM-Port nicht oeffnen. Falscher Port?";
39    }
40  }
41
42  /**
43   * @brief Hauptschleife der Benutzeroberflaeche. Steuert die Zustandsmaschine.
44   */
45  void draw() {
46    if (systemState.equals("ERROR")) {
47      drawErrorScreen();
48    }
49    else if (systemState.equals("TESTING")) {
50      drawTestingScreen();
51    }
52    else if (systemState.equals("OK")) {
53      drawRadarScreen();
54    }
55  }
56
57  // --- ZEICHNUNGS-FUNKTIONEN ---
58
59  /**
60   * @brief Zeichnet das Radarraster, die Scan-Linie und erkannte Hindernisse.
61   */
62  void drawRadarScreen() {
63    // Simuliert das Nachleuchten eines Radars
64    fill(0, 0, 0, 15);

```

```

65 noStroke();
66 rect(0, 0, width, height);
67
68 // Nullpunkt in die Mitte unten verschieben
69 pushMatrix();
70 translate(width/2, height - 20);
71
72 // 1. Raster zeichnen
73 strokeWeight(1);
74 stroke(0, 150, 0); // Dunkelgruen
75 noFill();
76
77 // Halbkreise (40cm, 30cm, 20cm, 10cm)
78 arc(0, 0, radarRadius * 2, radarRadius * 2, PI, TWO_PI);
79 arc(0, 0, radarRadius * 1.5, radarRadius * 1.5, PI, TWO_PI);
80 arc(0, 0, radarRadius, radarRadius, PI, TWO_PI);
81 arc(0, 0, radarRadius * 0.5, radarRadius * 0.5, PI, TWO_PI);
82
83 // Linien fuer die Winkel zeichnen
84 for (int a = 0; a <= 180; a += 30) {
85     float x = radarRadius * cos(radians(a));
86     float y = -radarRadius * sin(radians(a));
87     line(0, 0, x, y);
88 }
89
90 // --- BESCHRIFTUNG DES RASTERS ---
91 fill(0, 180, 0);
92
93 // A) Distanzen an der mittleren Achse eintragen
94 textAlign(LEFT, BOTTOM);
95 textSize(14);
96 text("10cm", 5, -(radarRadius * 0.25));
97 text("20cm", 5, -(radarRadius * 0.5));
98 text("30cm", 5, -(radarRadius * 0.75));
99 text("40cm", 5, -radarRadius);
100
101 // B) Winkelzahlen am aeusseren Rand eintragen
102 textSize(16);
103 for (int a = 0; a <= 180; a += 30) {
104     float textX = (radarRadius + 20) * cos(radians(a));
105     float textY = -(radarRadius + 20) * sin(radians(a));
106
107     if (a == 0) {
108         textAlign(LEFT, CENTER);
109     } else if (a == 180) {
110         textAlign(RIGHT, CENTER);
111     } else {
112         textAlign(CENTER, BOTTOM);
113     }
114     text(a + "°", textX, textY);
115 }
116
117 // 2. Aktuelle Scan-Linie zeichnen
118 strokeWeight(4);
119 stroke(0, 255, 0); // Hellgruen
120 float lineX = radarRadius * cos(radians(angle));
121 float lineY = -radarRadius * sin(radians(angle));
122 line(0, 0, lineX, lineY);
123
124 // 3. Hindernisse einzeichnen
125 if (distance < maxDistanceCm && distance > 0) {
126     // Pixel-Distanz berechnen
127     float pixDistance = map(distance, 0, maxDistanceCm, 0, radarRadius);
128     float objX = pixDistance * cos(radians(angle));
129     float objY = -pixDistance * sin(radians(angle));
130
131     fill(255, 0, 0); // Rot
132     noStroke();
133     ellipse(objX, objY, 15, 15);

```

```

134     }
135     popMatrix();
136
137     // --- WARNMELDUNG OBEN RECHTS ---
138     if (distance < maxDistanceCm && distance > 0) {
139         // Blink-Effekt
140         if (frameCount % 60 < 30) {
141             fill(255, 0, 0);
142         } else {
143             fill(100, 0, 0);
144         }
145
146         stroke(255);
147         strokeWeight(2);
148         rect(width - 280, 15, 260, 50, 5);
149
150         fill(255);
151         textAlign(CENTER, CENTER);
152         textSize(22);
153         text("OBJEKT_ERKANNT!", width - 150, 40);
154     }
155
156     // 4. UI Text Overlay links
157     fill(0, 0, 0, 255);
158     noStroke();
159     rect(15, 15, 260, 95);
160
161     stroke(0, 150, 0);
162     strokeWeight(1);
163     noFill();
164     rect(15, 15, 260, 95);
165
166     fill(0, 255, 0);
167     textAlign(LEFT, TOP);
168     textSize(20);
169     text("Status: SYSTEM_OK", 20, 20);
170     text("Winkel: " + int(angle) + "°", 20, 50);
171
172     if (distance < maxDistanceCm && distance > 0) {
173         text("Distanz: " + int(distance) + " cm", 20, 80);
174     } else {
175         text("Distanz: Ausser Reichweite", 20, 80);
176     }
177 }
178
179 /**
180  * @brief Zeigt den Ladebildschirm waehrend der Kalibrierung des Arduinos.
181  */
182 void drawTestingScreen() {
183     background(30);
184     fill(255, 200, 0);
185     textAlign(CENTER, CENTER);
186     textSize(36);
187     text("System_faehrt_hoch...", width/2, height/2 - 20);
188     textSize(20);
189     text("Hardware-Selbsttest_wird_durchgefuehrt.", width/2, height/2 + 30);
190 }
191
192 /**
193  * @brief Zeigt einen Fehlerbildschirm an (z.B. fehlender COM-Port oder Sensor-Error).
194  */
195 void drawErrorScreen() {
196     background(150, 0, 0);
197     fill(255);
198     textAlign(CENTER, CENTER);
199     textSize(50);
200     text("SYSTEMFEHLER", width/2, height/2 - 60);
201
202     textSize(24);

```

```

203     text(errorMessage, width/2, height/2 + 20);
204
205     if (frameCount % 60 < 30) {
206         fill(255, 255, 0);
207         text("Anlage_ueberpruefen_und_Arduino_neustarten.", width/2, height/2 + 80);
208     }
209 }
210
211 // --- SERIELLE KOMMUNIKATION ---
212
213 /**
214  * @brief Interrupt-Funktion. Wird aufgerufen, sobald serielle Daten eintreffen.
215  * @param p Das Serial-Objekt, welches die Daten empfaengt.
216  */
217 void serialEvent(Serial p) {
218     try {
219         String incoming = p.readStringUntil('\n');
220         if (incoming != null) {
221             incoming = trim(incoming);
222             String[] parts = split(incoming, ':');
223
224             if (parts.length == 2) {
225                 String type = parts[0];
226                 String value = parts[1];
227
228                 // Status verarbeiten
229                 if (type.equals("S")) {
230                     if (value.equals("OK")) systemState = "OK";
231                     else if (value.equals("TESTING")) systemState = "TESTING";
232                     else if (value.equals("ERR_SENSOR")) {
233                         systemState = "ERROR";
234                         errorMessage = "Ultraschallsensor_antwortet_nicht";
235                     }
236                 }
237                 // Radar-Daten verarbeiten
238                 else if (type.equals("D") && systemState.equals("OK")) {
239                     String[] data = split(value, ',');
240                     if (data.length == 2) {
241                         angle = float(data[0]);
242                         distance = float(data[1]);
243                     }
244                 }
245             }
246         }
247     } catch (Exception e) {
248         println("Fehler_beim_Lesen_der_Schnittstelle");
249     }
250 }

```


Teil II.

Domain Knowledge - Hardware

3. Arduino Nano 33 BLE Sense Lite

Arduino ist eine Open-Source-Elektronikplattform, die auf leicht zugänglicher Hardware und Software basiert. Sie stellt Mikrocontroller- und Mikroprozessorkits bereit, mit denen sich sowohl digitale als auch analoge Anwendungen realisieren lassen. Der Mikrocontroller oft als eingebetteter Mini-Computer auf dem Arduino-Board bezeichnet benötigt üblicherweise zusätzliche elektronische Bauteile wie Dioden, Widerstände, Kondensatoren oder Transistoren, um Spannungen und Ströme korrekt zu steuern.

Das Arduino-Team hat jedoch eine benutzerfreundliche Umgebung geschaffen, die viele dieser elektronischen Komplexitäten abstrahiert. Statt komplexer Schaltungen genügt es, das Board mit der vorgesehenen Spannung zu versorgen, das gewünschte Programm zu schreiben und es innerhalb weniger Sekunden auf den Mikrocontroller zu übertragen. Dadurch wird die gesamte Hardwarefunktionalität durch Softwaresteuerung zugänglich gemacht, ohne tiefgreifende Elektronikkenntnisse zu erfordern.

Durch den technologischen Fortschritt in der Halbleiter- und Elektronikindustrie lassen sich viele Steuerungsaufgaben heute mithilfe kompakter Mikrocontroller lösen anstelle mechanischer oder elektromechanischer Schalter. Allen Arduino-Boards ist gemeinsam, dass sie einen Mikrocontroller enthalten einen kleinen, programmierbaren Rechner, der sich ideal für Anwendungen im Bereich des Edge Computings eignet

3.1. Arduino Nano 33 BLE Sense

Die Arduino Nano-Familie umfasst eine Reihe kompakter Boards mit vielfältigen Funktionen vom preisgünstigen Einstiegsmodell bis hin zu leistungsfähigeren Varianten wie dem Nano 33 BLE Sense oder dem Nano RP2040 Connect, die mit integrierten Bluetooth- bzw. Wi-Fi-Modulen ausgestattet sind. Darüber hinaus verfügen diese Boards über eine Reihe eingebetteter Sensoren, etwa für Temperatur, Luftfeuchtigkeit, Luftdruck, Gestenerkennung, Geräuscherfassung und weitere Umgebungsparameter. Sie unterstützen sowohl die Programmierung in C/C++ als auch in MicroPython und ermöglichen die Einbindung von Machine-Learning-Anwendungen.

Das Arduino Nano 33 BLE Sense ist ein leistungsfähiges Modell dieser Serie. Es kombiniert Bluetooth Low Energy (BLE) Kommunikation mit einer umfangreichen Sensorausstattung und basiert auf dem NINA B306 Modul, das für BLE- und Bluetooth 5.0-Verbindungen ausgelegt ist dem aktuellen Standard für energiesparende drahtlose Kommunikation. Als zentraler Mikrocontroller kommt ein Nordic nRF52840 zum Einsatz, ausgestattet mit einem Cortex-M4F-Prozessor. Aufgrund seiner kompakten Bauform, der reduzierten Leistungsaufnahme und seiner umfangreichen Sensorik eignet sich das Board besonders für energieeffiziente, interaktive und tragbare Anwendungen.

Die Modellbezeichnung Arduino Nano 33 BLE Sense verweist bereits auf zentrale Eigenschaften: Die Bezeichnung Nano beschreibt das kompakte Format des Boards, 33 steht für die Betriebsspannung von 3,3 Volt, BLE kennzeichnet die integrierte Bluetooth-Low-Energy-Funktionalität, und Sense weist auf die Vielzahl integrierter Sensoren hin, darunter Beschleunigungs-, Lage-, Magnetfeld-, Temperatur-, Feuchtigkeits- und Luftdrucksensoren, ein Näherungs-, Farb- und Gestensensor sowie ein eingebettetes

Mikrofon.

Für Anwendungen im Bereich Objekterkennung, Farbanalyse oder Gestensteuerung kann das Arduino Nano 33 BLE Sense mit dem Kamera-Shield Arducam OV2640 kombiniert werden. Dieses Kameramodul erweitert die Sensorik des Boards um Bildverarbeitungsfunktionen und eignet sich besonders für KI- und Machine-Learning-Anwendungen. Durch die Installation geeigneter Bibliotheken lassen sich alle Sensoren und das Kameramodul direkt über die Arduino-Entwicklungsumgebung ansprechen und in intelligente Systeme integrieren.

Die folgende Abbildung 3.1 zeigt einen Arduino Nano 33 BLE Sense Revision 2, zu sehen ist die kompakte Bauform.

Abbildung 3.1.: Arduino Nano 33 BLE Sense Rev. 2, see Arduino Store

Das kompakte und zuverlässige Arduino Nano 33 BLE Sense Board basiert auf dem NINA B306 Modul, das für Bluetooth Low Energy (BLE) und Bluetooth 5 Kommunikation ausgelegt ist. Dieses Modul wiederum verwendet den Nordic nRF52840 Prozessor, der mit einem leistungsfähigen Cortex-M4F-Kern ausgestattet ist. Die Architektur des Boards ist vollständig kompatibel mit der Arduino IDE, sowohl in der Online- als auch in der Offline-Version. Das Arduino Nano 33 BLE Sense integriert eine Vielzahl von Sensoren auf kleinstem Raum und eignet sich daher hervorragend für kompakte, sensorbasierte Anwendungen.

Zu den auf dem Board verbauten Sensoren gehören der ADPS-9960, der Umgebungslicht, RGB-Farbe, Näherung und Gesten erfassen kann, sowie der LSM9DS1, der als System-in-Package eine dreiaxige Beschleunigungserfassung, ein Gyroskop und ein Magnetometer kombiniert. Der LPS22HB misst den Luftdruck und die Umgebungstemperatur, während der HTS221 für die Erfassung der relativen Luftfeuchtigkeit zuständig ist. Zusätzlich ist mit dem MP34DT05-A ein digitales Mikrofon integriert, das Audiosignale erfassen kann. Die Bluetooth-Kommunikation wird über das bereits erwähnte NINA B306 Modul abgewickelt, wodurch drahtlose Datenübertragung mit geringem Energieverbrauch möglich wird. Mit dieser umfangreichen Ausstattung lassen sich komplexe IoT- und KI-Projekte auf kleinstem Raum realisieren.

3.1.1. Arduino Nano 33 BLE Sense Lite

4. Arduino Tiny Machine Learning Kit

Das Arduino Tiny Machine Learning Kit ist eine leistungsfähige Entwicklungsplattform, die es ermöglicht, Machine-Learning-Modelle direkt auf energieeffizienter Mikrocontroller-Hardware auszuführen. Im Mittelpunkt des Kits steht das Arduino Nano 33 BLE Sense Lite Board, das eine Vielzahl integrierter Sensoren für Beschleunigung, Temperatur, Luftdruck, Geräusche, Gesten und vieles mehr beinhaltet. Enthalten sind:

- Arduino Nano 33 BLE Sense Lite
- Kameramodul (OV7675)
- Arduino Tiny Machine Learning Shield
- USB-A-auf-USB-Micro-Verbindungskabel

4.1. Arduino Nano 33 BLE Sense Lite

Das Arduino Nano 33 BLE Sense ist ein kompakt gebautes Entwicklungsboard, das mit dem NINA B306 Funkmodul ausgestattet ist. Dieses Modul basiert auf dem Nordic nRF52840 Mikrocontroller und nutzt einen leistungsstarken Arm Cortex-M4F Prozessor. Zusätzlich verfügt es über einen integrierten Kryptografie-Chip, der eine sichere Speicherung von Zertifikaten und vorab geteilten Schlüsseln ermöglicht. Für Anwendungen im Bereich Bewegungserkennung bietet das Board außerdem eine 9-Achsen-Inertialsensorik (IMU). Die Platine kann flexibel entweder als steckbares Bauteil mit montierten Stiftleisten oder durch direktes Verlöten der castellierten Kontaktflächen auf einer Leiterplatte verwendet werden. Das Arduino Nano 33 BLE Sense besitzt, wie alle Modelle der Nano-Serie, kein integriertes Ladesystem. Die Energieversorgung erfolgt wahlweise über die USB-Schnittstelle oder über die an den Stiftleisten verfügbaren Versorgungsanschlüsse. [Arduino:2025]

Zu beachten ist, dass dieses Board ausschließlich mit 3,3-Volt-Pegeln an seinen Ein- und Ausgängen arbeitet. Eine direkte Verbindung von 5-Volt-Signalen kann die Schaltung irreparabel beschädigen. Anders als bei älteren Arduino Nano-Versionen, bei denen der 5V-Pin eine stabile 5V-Versorgung bereitstellt, ist dieser Anschluss beim Nano 33 BLE Sense nicht als Spannungsquelle nutzbar. Er ist lediglich intern mit dem USB-Eingang verbunden und führt nur dann Spannung, wenn das Board per USB mit Strom versorgt wird. [Arduino:2025]

4.1.1. Pinbelegung

Die exakte Zuordnung der verfügbaren Pins des Arduino Nano 33 BLE Sense wird in Abbildung ?? grafisch dargestellt. Eine detaillierte Übersicht über die Funktionen, Signaltypen und Anschlussmöglichkeiten der einzelnen Pins liefert Tabelle 4.1. Diese Informationen sind essenziell für die korrekte Anbindung externer Komponenten und die Planung von Schaltungen.

Tabelle 4.1.: Pinbelegung des Arduino Nano 33 BLE Sense [Arduino:2025]

PIN	Funktion	Typ	Beschreibung
1	D13	Digital	GPIO

PIN	Funktion	Typ	Beschreibung
2	+3V3	Power Out	Intern erzeugte Stromausgabe an externe Geräte
3	AREF	Analog	Analoge Referenz; kann als GPIO verwendet werden
4	A0/DAC0	Analog	ADC-Eingang/DAC-Ausgang, kann als GPIO verwendet werden
5	A1	Analog	ADC-Eingang; kann als GPIO verwendet werden
6	A2	Analog	ADC-Eingang; kann als GPIO verwendet werden
7	A3	Analog	ADC-Eingang; kann als GPIO verwendet werden
8	A4/SDA	Analog	ADC-Eingang; I2C SDA; kann als GPIO verwendet werden
9	A5/SCL	Analog	ADC-Eingang; I2C SCL; kann als GPIO verwendet werden
10	A6	Analog	ADC-Eingang; kann als GPIO verwendet werden
11	A7	Analog	ADC-Eingang; kann als GPIO verwendet werden
12	VUSB	Power In/Out	Normalerweise NC; kann durch Kurzschließen eines Jumpers an den VUSB-Pin des USB-Anschlusses angeschlossen werden
13	RST	Digital In	Aktiv-Low-Reset-Eingang (Duplikat von Pin 18)
14	GND	Power	Strom Masse
15	VIN	Power In	VIN Stromeingang
16	TX	Digital	USART TX; Kann als GPIO verwendet werden
17	RX	Digital	USART RX; Kann als GPIO verwendet werden
18	RST	Digital	Aktiv-Low-Reset-Eingang (Duplikat von Pin 13)
19	GND	Power	Strom Masse
20	D2	Digital	GPIO
21	D3/PWM	Digital	GPIO; kann als PWM verwendet werden
22	D4	Digital	GPIO
23	D5/PWM	Digital	GPIO; kann als PWM verwendet werden
24	D6/PWM	Digital	GPIO; kann als PWM verwendet werden
25	D7	Digital	GPIO
26	D8	Digital	GPIO
27	D9/PWM	Digital	GPIO; kann als PWM verwendet werden
28	D10/PWM	Digital	GPIO; kann als PWM verwendet werden
29	D11/MOSI	Digital	SPI MOSI; kann als GPIO verwendet werden
30	D12/MISO	Digital	SPI MISO; kann als GPIO verwendet werden

4.1.2. Mechanik

Formfaktor: Der Arduino Nano 33 BLE Sense hat einen kleinen Formfaktor, der typischerweise etwa 45mm x 18mm misst, wobei Toleranzen von etwa +/- 1mm

möglich sind. Montagemöglichkeiten: Die Platine verfügt über Bohrungen mit einem Durchmesser von etwa 3mm, wobei Toleranzen von $\pm 0,5\text{mm}$ möglich sind.

4.1.3. Elektronik

Prozessoreinheit: Der Arduino Nano 33 BLE Sense wird von einem 64 MHz Arm Cortex-M4F Prozessor angetrieben, wobei die Taktfrequenz um bis zu $\pm 5\text{ MHz}$ variieren kann. **NINA B306 Modul:** Das integrierte NINA B306 Modul bietet Bluetooth 5-Konnektivität mit einer Sendeleistung von $+8\text{ dBm}$ und Empfindlichkeit von -95 dBm , wobei Toleranzen von $\pm 1\text{ dB}$ möglich sind.

4.1.4. Verbaute Sensoren

LSM9DS1 - Trägheitssensor: Die Sensordaten haben eine Genauigkeit von etwa $\pm 5\text{ Prozent}$ und eine Prozentual mögliche Messbereichs toleranz von $\pm 2\text{ Prozent}$.

LPS22HB - Drucksensor: Der Drucksensor hat eine Genauigkeit von etwa $\pm 0,5\text{ hPa}$ und eine Temperaturgenauigkeit von etwa $\pm 0,2^\circ\text{C}$.

HTS221 - Feuchtigkeitssensor: Die relative Feuchtigkeitsmessung hat eine Genauigkeit von etwa $\pm 5\text{ Prozent}$ und eine Temperaturgenauigkeit von etwa $\pm 0,5^\circ\text{C}$.

APDS-9960 - Näherungs- und RGB-Sensor: Die Näherungssensorgenauigkeit beträgt etwa $\pm 2\text{ mm}$, die RGB-Farberkennung hat eine Genauigkeit von etwa $\pm 5\text{ Prozent}$.

MP34DT05 - Digitales Mikrofon: Die Genauigkeit des digitalen Mikrofons beträgt etwa $\pm 2\text{ dB}$.

ATECC608A - Kryptografischer Co-Prozessor: Die kryptografische Funktion hat eine Genauigkeit von etwa $\pm 1\text{ Bit}$.

MPM3610 - DC-DC-Wandler: Die Effizienz des DC-DC-Wandlers kann um bis zu $\pm 5\text{ Prozent}$ variieren.

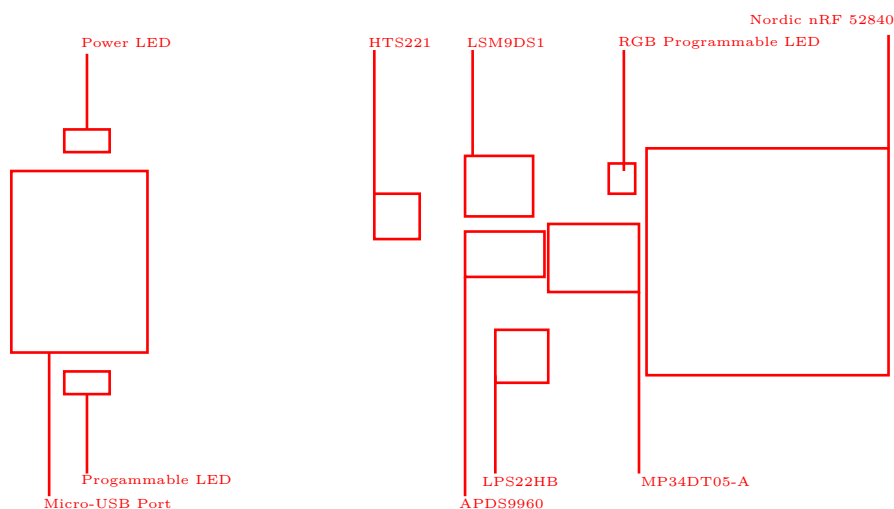


Abbildung 4.1.: Arduino Nano 33 BLE Sense Architektur

The Tiny Machine Learning Kit contains only the Arduino Nano 33 BLE Sense Lite.

The Arduino Nano 33 BLE Sense Lite differs only slightly from the Arduino Nano 33 BLE Sense Revision 1. The only difference between the two models is that the Lite version does not have the HTS221, the sensor for relative humidity and temperature. The LPS22HB sensor, which is on the board, is a pressure sensor, but it can also be used to measure the temperature. It is therefore not possible to measure humidity with the Arduino Nano 33 BLE Lite. To measure relative humidity, a separate sensor must be connected. The reason for this decision is that the Arduino company has difficulties maintaining the stock. [Filipi:2022]

The figure ?? presents the layout of the Arduino Nano 33 BLE Lite,

4.2. On-Board Sensor Description

Arduino Nano 33 BLE sense come up with the set of embed sensor on the board. The available embed sensors are commonly use for measuring both the analog and digital values around the surrounding. Arduino Nano 33 BLE sense is very small 45mm × 18mm in size, which makes it very usefull for Internet of things (IOT) and Artificial intelligence (AI) application as a embed device where space is the main constrained issue. It is low power consumption board and operate normally on 3.3 V, we can say that this small size low power consumption board can operate on small batteries even for many months. Due to on-board available sensor, the low power consumption and mini architecture we can use this nano board anywhere. The Arduino Nano 33 BLE Sense is a completely new board on a well-known form factor. For getting detail information about each component of Arduino Nano 33 BLE Sense and data sheets of each sensor the following links give us a detail information [Arduino:2021]. The short description of each sensor are as follow.

- The ADPS-9960 is a digital proximity, ambient light, RGB and gesture sensor. it can measure the proximity distance, light, color and gestures when moving close with the borad.
- The sensor LSM9DS1 is a 9 axis Inertialmesseinheit (IMU) use as a accelerometre, gyroscope, and magnatometre, this 9 axis sensor is ideal for wearable devices.
- The sensor LPS22HB is a barometric pressure sensor, it measures the environmental pressure which is usefull for simple weather station monitoring.
- The sensor HTS221 senses the relative humidity, and temperature, to get highly accurate measurements of the environmental conditions.
- The sensor MP34DT05 is the digital microphone. it is usefull for capturing, analyzing and detecting the sound in real time.
- The USB port allows you to connect Arduino Nano 33 BLE sense to your machine.
- There are 3 different LEDs that can be accessed on the Nano BLE Sense: RGB Programmable LED , the built-in orange Programmable LED and the Power LED.

The figure 4.2 show the embed sensors on the board, with powerful processor as compared to other arduino boards the nRF52840 from Nordic Semiconductors, a 32-bit ARM® Cortex™-M4 CPU processor running at 64 MHz are as follow.

MICROCONTROLLER	nRF52840
OPERATING VOLTAGE	3.3V
INPUT VOLTAGE (LIMIT)	21V
DC CURRENT PER I/O PIN	15 mA
CLOCK SPEED	64MHz
CPU FLASH MEMORY	1MB
SRAM	256KB
LED_BUILTIN	13
IMU (Accelerometer, Gyroscope, Magnetometer)	LSM9DS1
MICROPHONE	MP34DT05
GESTURE, LIGHT, PROXIMITY, COLOUR	APDS9960
BAROMETRIC PRESSURE	LPS22HB
TEMPERATURE, HUMIDITY	HTS221

Tabelle 4.2.: Technical Specifications of Arduino Nano 33 BLE Sense [Arduino:2021]

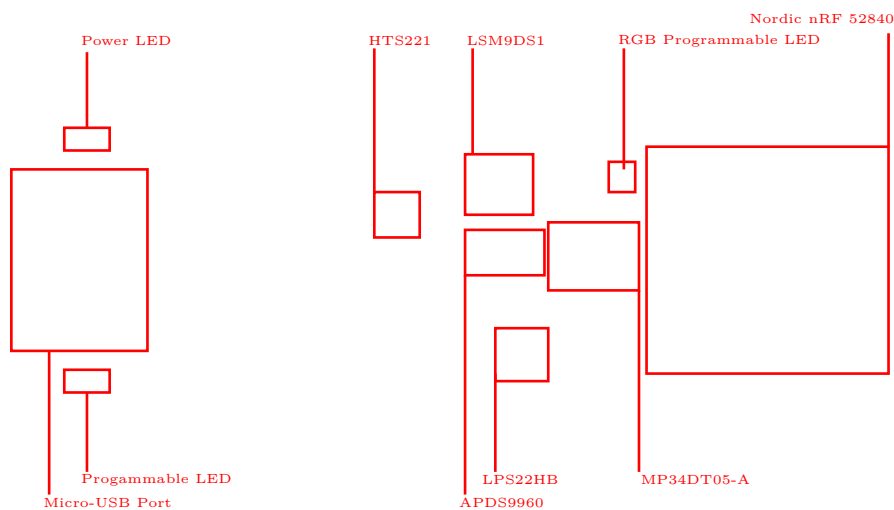


Abbildung 4.2.

WS:Auch mit dem Rev 2

Another point to bear in mind is the overall 'trueness' of sensor readings based on where do you place the actual sensors in the environment, as this could be quite critical. The operating temperature should not exceed 85°C and not lower than -40°C . Other factors like humidity level and air pressure values should also be kept in check.

(15)	~D2/MISO	(16)	SCK/~D13
(14)	~D2/MOSI	(17)	+3V3
(13)	~D10	(18)	AREF
(12)	~D9	(19)	A0
(11)	~D8	(20)	A1
(10)	~D7	(21)	A2
(9)	~D6	(22)	A3
(8)	~D5	(23)	SDA/A4
(7)	~D4	(24)	SCL/A5
(6)	~D3	(25)	A6
(5)	~D2	(26)	A7
(4)	GND	(27)	+5V
(3)	Reset	(28)	Reset
(2)	D0/RX	(29)	GND
(1)	D1/TX	(30)	V_{in}

Abbildung 4.3.: Pin assignment of the Arduino Nano 33 BLE Sense and for the Arduino Nano 33 BLE Sense Rev 2; note that the orange built-in LED is connected to pin D13 and the power LED to pin D1. The built-in RGB LED occupies pins D18, D19 and D20.

WS:Beide Diagramme

4.2.1. Gesture, Proximity, and Color Detection Sensor ADPS-9960

The APDS-9960 device features advanced Gesture detection, Proximity detection, Digital Ambient Light Sense (ALS) and Color Sense (RGB). [Arduino:2021] Gesture detection utilizes four directional photodiodes to sense reflected IR energy (sourced by the integrated LED) to convert physical motion information (i.e. velocity, direction and distance) to a digital information.

For the following Applications the sensor is in use:

- Gesture Detection
- Color Sense
- Ambient Light Sensing
- Proximity Sensing

4.2.2. Accelerometer, Gyroscope, and Magnetometre Sensor LSM9DS1

The LSM9DS1 is a system-in-package featuring a 3D digital linear acceleration sensor, a 3D digital angular rate sensor, and a 3D digital magnetic sensor. IMU's work by detecting rotational movements across the 3 axis known as Pitch, Roll and Yaw. To

achieve the same, it depends on Accelerometer, Gyroscope and Magnetometer. The accelerometer gives the velocity at which the IMU module moves. The gyroscope measure the rotational movement rate on the IMU. Magnetometer measures the force of gravity acting on the IMU.

For the following Applications the IMU is in use:

- Indoor navigation
- Advanced gesture recognition
- Gaming and virtual reality input devices
- Display/map orientation and browsing
- Consumer electronics; Smartphones, tablets, fitness trackers for motion sensing and orientation.
- Compact transportation solutions like Segway.
- Sports Technology - helping athletes to know how they can improve their movements.

There are also certain disadvantages of using the IMU and points that need to be kept in mind while using an IMU sensor. Accumulated error or '*Drift*' is the main disadvantage of IMUs, present due to its constant measuring of changes and rounding off its calculated values off. When such a process happens for a prolonged period of time, it can lead to significant errors. The best way to avoid the *drift* factor is to use a good quality IMU Sensor and make sure the IMU sensor is calibrated. [STMicroelectronics:2015]

Calibration of an IMU

On research it was found that there are various methods to calibrate the sensors involved, the time period between each calibration is also not defined specifically, however it is advised that regular calibration is done especially when there are strange outputs noticed. Few methods of calibration are briefed below:

Low and High Limit Method

In this method the sensor is rotated in circles along each axis a few times. The midpoint is then found between the two extremes. If there is no offset, the midpoint is close to zero, but if there is a slight deviation from zero, this figure is the hard iron offset, which is the result of the distortion caused by the Earth's magnetic field. This method is mainly used to calibrate the Magnetometer [Mallon:2015]

Magneto V1.2

In this method, the raw magnetometer data is pre-processed with axis specific gain correction to convert the raw output into nanoTesla:

```
Xm_nanoTesla = rawCompass.m.x*(100000.0/1100.0);
Gain X [LSB/Gauss] for selected input field range
Ym_nanoTesla = rawCompass.m.y*(100000.0/1100.0);
Zm_nanoTesla = rawCompass.m.z*(100000.0/980.0);
```

This converted data is saved into the file `Mag_raw.txt` that you open with the Magneto program. To start using this method, we first need to replace the (100000.0/1100.0) scaling factors with values that convert your specific sensors output into nanoTesla.

Rather than simply finding an offset and scale factor for each axis, Magneto creates twelve different calibration values that correct for a whole set of errors: bias, hard iron, scale factor, soft iron and misalignment.

A side benefit of this is that it can be used to calibrate accelerometers as well. You might again need to pre-process your specific raw accelerometer output, taking into account the bit depth and G sensitivity, to convert the data into milliGalileo. Then enter a value of 1000 milliGalileo as the “norm” for the gravitational field. [Mallon:2015]

WS: Here, we need more information because we

4.2.3. Pressure Sensor LPS22HB

The LPS22HB is an ultra-compact piezoresistive absolute pressure sensor which functions as a digital output barometer.

For the following Applications the sensor is in use:

- Altimeters and barometers for portable devices
- Weather station equipment
- Sports watches

A sensor element is installed as well as an IC interface that communicates via an I²C or SPI bus. The function is given in a temperature range from -40°C to $+85^{\circ}\text{C}$. [STM17]

- Absolute Druckbereich: 260 bis 1260 hPa
- Versorgungsspannung: 1,7 bis 3,6 Volt
- 24-bit Druckdatenausgabe
- 16-bit Temperaturdatenausgabe

4.2.4. Relative Humidity and Temperature Sensor HTS221

The HTS221 is an ultra-compact sensor for relative humidity and temperature. It includes a sensing element and a mixed signal to provide the measurement information through digital serial interfaces.

For the following Applications the sensor is in use:

- Air conditioning, heating and ventilation
- Air humidifiers
- Refrigerators
- Smart home automation
- Industrial automation

Dieser Sensor kommuniziert über den I²C- und SPI-Bus. Dieser Sensor ist einsetzbar in einem Temperaturbereich von -40°C bis $+120^{\circ}\text{C}$. Zur Spannungsversorgung werden 1,7 bis 3,3 Volt benötigt. Eine Temperaturmessung erfolgt mit einer Genauigkeit von $\pm 5^{\circ}\text{C}$. [STM23]

- GND - Ground
- DRDY - Data ready output signal
- SCL/SPC - I2C serial clock (SCL) & SPI serial port clock (SPC)

- VDD - Stromversorgung
- SDA/SDI/SDO - I²C serial Data (SDA) & 3 wire-SPI serial data input/output (SDI/SDO)
- SPI enable - I²C/SPI mode selection

4.2.5. Digital Microphone MP34DT05-A

The MP34DT05-A is an ultra-compact, low-power, omni directional, digital microphone built with a capacitive sensing element and an IC interface. The sensing element, capable of detecting acoustic waves, is manufactured using a specialized silicon micromachining process dedicated to producing audio sensors. The MP34DT05 is a low-distortion microphone with a signal-to-noise ratio of 64 dB. The sensitivity is -26 dBFS $\pm$ 3 dB. The Acoustic Overload Point (AOP) is 122.5 dBSPL.

The output signal of the microphone is a PDM signal. This signal is a binary signal modulated by Pulse Density Modulation (PDM) from the analogue signal. [STMicroelectronics:2021]

For the following Applications the sensor is in use:

- Speech recognition
- Portable media player
- Mobile Terminal

The Arduino nano 33 BLE Sense has a built-in microphone that uses PDM (Pulse-Density Modulation) to convert sound into digital data. You can use the PDM library to access the microphone data and perform various tasks with it. Here is a simple example of using the builtin microphone for an Arduino nano 33 BLE Sense:

The code 4.1 will print the sound level of the microphone to the serial monitor every 100 milliseconds.

You can modify this code to perform other tasks with the microphone data, such as controlling LEDs, playing sounds, or sending data to other devices. For more information and examples, you can check out the PDM library documentation or the Arduino nano 33 BLE Sense page.

4.2.6. Bluetooth Module nRF52840

The nRF52840 is an advanced, highly flexible single chip solution for today's increasingly demanding Ultra Low Power (ULP) wireless applications for connected devices on our person, connected living environments and the Internet of Things (IoT) at large. It is designed ready for the major feature advancements of Bluetooth 5 and takes advantage of Bluetooth 5's increased performance capabilities. [Arduino:2021b]

Applications

- Smart Home products
- Industrial mesh networks
- Smart city infrastructure
- Connected watches
- Advanced personal fitness devices
- Wearables with wireless payment
- Connected Health

```

1000 // Include the PDM library
1001 #include <PDM.h>
1002
1003 // Buffer to store the microphone data
1004 short sampleBuffer[256];
1005
1006 // Variable to store the sound level
1007 int soundLevel = 0;
1008
1009 // Callback function for PDM data
1010 void onPDMdata() {
1011     // Read the PDM data
1012     int bytesAvailable = PDM.available();
1013     PDM.read(sampleBuffer, bytesAvailable);
1014
1015     // Calculate the sound level
1016     soundLevel = 0;
1017     for (int i = 0; i < bytesAvailable / 2; i++) {
1018         soundLevel += abs(sampleBuffer[i]);
1019     }
1020     soundLevel /= bytesAvailable / 2;
1021 }
1022
1023 void setup() {
1024     // Initialize serial communication
1025     Serial.begin(9600);
1026     while (!Serial);
1027
1028     // Initialize PDM with a sample rate of 16 kHz and 16-bit resolution
1029     PDM.begin(1, 16000);
1030     PDM.onReceive(onPDMdata);
1031 }
1032
1033 void loop() {
1034     // Print the sound level to the serial monitor
1035     Serial.println(soundLevel);
1036     delay(100);
1037 }

```

Listing 4.1.: Simple example using of the builtin microphone of the Arduino Nano 33 BLE Sense

4.3. Bluetooth Module INA B306

The module is a Bluetooth Low Energy (BLE). The connection with our smartphone is possible, to do this we have to use application “Nordic Semiconductors nRF Connect” They used 5.0 generation bluetooth. ??

We can use for example to share Arduino batterie on smartphone, ?? here you see an code example for how we can do this.

4.4. Arduino Nano 33 BLE Pin Configuration

Arduino Nano 33 BLE is an advanced version of Arduino Nano board that is based on a powerful processor the nRF52840. The figure ?? shows that the board has the following pin configuration. Pin Configuration

Digital pin The number of digital I/O pins are 14 which receive only two values HIGH or LOW. These pins can either be used as an input or output based on the requirement. When these pins receive 5V, they are in a HIGH state and when they receive 0V they are in a LOW state.

Analog pin Total 8 analog pins available on the board A0 – A7. These pins get any value as opposed to digital pins that only receive two values HIGH or LOW. These pins are used to measure the analog voltage ranging between 0 to 5V.

PWM pin All digital pins can be used as PWM pins. These pins generate analog results with digital means.

SPI pin The board supports serial peripheral interface (SPI) communication protocol. This protocol is employed to develop communication between a controller and other peripheral devices like shift registers and sensors. Two pins are used for SPI communication i.e. Master Input Slave Output (MISO) and Master Output Slave Input (MOSI) are used for SPI communication. These pins are used to send or receive data by the controller.

I2C pin The board carries the I2C communication protocol which is a two-wire protocol. It comes with two pins SDL and SCL.

UART pin The board features a UART communication protocol that is used for serial communication and carries two pins Rx and Tx. The Rx is a receiving pin used to receive the serial data while Tx is a transmission pin used to transmit the serial data.

External Interrupts pin All digital pins can be used as external interrupts. This feature is used in case of emergency to interrupt the main running program with the inclusion of important instructions at that point.

LED at Pin 13 and AREF pin There is an LED connected to pin 13 of the board. And AREF is a pin used as a reference voltage for the input voltage.

4.5. Was fehlt

- Beschreibung der Sensoren
 - Hintergrundwissen
 - Beschreibung
 - Eigenschaften

- Kalibrierung
- Beschreibung der Bibliothek
 - Beschreibung
 - Installation
 - Funktionen
- Laufzeit, Speicherplatz, ...
- Software-Dokumentation
- Verwendung von Werkzeugen, z.B. Doxygen
 - Beschreibung, inklusive im Quellcode; z.B. Header
 - Handbuch
 - Ablaufdiagramm
 - ...
- Interpretation der Ergebnisse
- Literatur, Bilder
- ...

4.6. Further Readings

5. Ultraschallsensor HC-SR04

Einleitung

Dieses Kapitel befasst sich mit der berührungslosen Abstandsmessung mithilfe von Sensoren. Zunächst wird ein allgemeiner Überblick über Ultraschallsensoren zur Distanzmessung gegeben. Anschließend wird der in diesem Projekt verwendete Ultraschallsensor HC-SR04 detailliert beschrieben, einschließlich seiner technischen Spezifikationen und der softwareseitigen Implementierung unter Verwendung ausgewählter C++ Bibliotheken.

5.1. Allgemeines

Sensoren zur Abstandsmessung sind wichtige Komponenten in der modernen Automatisierungstechnik, Robotik und Fahrzeugtechnik [Fra16]. Sie ermöglichen es Systemen, eine Umgebung räumlich zu erfassen. Ultraschallsensoren arbeiten nach dem Echo-Lot-Prinzip (Schalllaufzeitmessung). Sie senden Schallwellen aus und messen die Zeit, bis das Echo zurückkehrt [Ele20]. Sie sind unabhängig von der Farbe oder optischen Transparenz eines Objekts, haben jedoch ihre Grenze bei schallabsorbierenden Materialien (z. B. Schaumstoff) [Fra16].

5.2. Ultraschallsensor HC-SR04

In diesem Projekt kommt der Ultraschallsensor **HC-SR04** zum Einsatz. Er wandelt die gemessene Entfernung in ein PWM-Signal um, welches am Ausgang zur Verfügung steht [KT-12]. Das Auslösen eines Messzyklus geschieht durch eine fallende Flanke am TRIG-Eingang für mindestens 10 μ s. Das Modul sendet darauf nach ca. 250 μ s ein 40 kHz Burst-Signal für die Dauer von 200 μ s aus. Danach geht der ECHO-Ausgang sofort auf HIGH-Pegel und das Modul wartet auf den Empfang des Echos. Wird dieses detektiert, fällt der Ausgang auf LOW-Pegel. Wird kein Echo detektiert, verweilt der Ausgang für insgesamt 200 ms auf HIGH-Pegel und zeigt so eine erfolglose Messung an. Ein vollständiges Messintervall hat eine Dauer von 20 ms, sodass maximal 50 Messungen pro Sekunde durchgeführt werden können [KT-12].

5.3. Spezifikation

Die elektrischen und physikalischen Parameter entstammen den offiziellen Spezifikationen von KT-elektronic [KT-12].

- Betriebsspannung (VCC: +5 V DC $\pm 10\%$)
- Stromaufnahme: < 2 mA
- Messbare Distanz: 2 cm bis ca. 300 cm
- Messauflösung: 3 mm
- Messintervall: 20 ms (maximal 50 Messungen pro Sekunde)

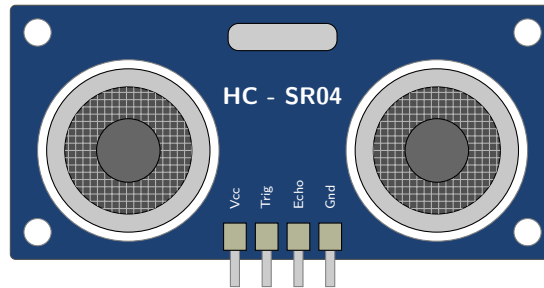


Abbildung 5.1.: Schematische Darstellung des Ultraschallsensors HC-SR04

- Messwinkel: Bis zu 45° bei kurzen Distanzen (< 1 m), Sendekegel von 15° bei maximaler Distanz
- Abmessungen (L x B x T): 45 x 21 x 18 mm

Schaltplan Das Modul besitzt vier Pins (siehe Abbildung 5.1). Der VCC-Pin ist mit der 5V-Spannungsversorgung des Mikrocontrollers verbunden, der GND-Pin (Pin 4) mit dem GND-Pin des Mikrocontrollers. Der TRIG-Eingang (Pin 2) und der ECHO-Ausgang (Pin 3) arbeiten mit TTL-Pegel (siehe Abbildung 5.2). Hinweis bei 3.3V-Mikrocontrollern: Da der ECHO-Pin ein 5V-Signal liefert, muss dieser zwingend durch einen Spannungsteiler an den 3.3V-Eingang des Mikrocontrollers angepasst werden [KT-12].

5.4. Bibliothek

5.4.1. Beschreibung

Für die Ansteuerung des Sensors wird die externe C++ Bibliothek **NewPing** von Tim Eckel verwendet [Eck23].

5.4.2. Installation

Die Bibliothek kann direkt über den internen Bibliotheksverwalter der Arduino IDE installiert werden [Eck23].

- **Data types:** Messzeiten in Mikrosekunden und Distanzen in Zentimetern werden als `unsigned int` deklariert, um Speicherplatz zu sparen und negative Distanzen auszuschließen.
- **Data structure:** Der Sensor wird als Objekt der Klasse `NewPing` instanziiert.
- **Data quantity:** Das Softwaredesign ist ressourcenschonend konzipiert. Es können bis zu 15 Ultraschallsensoren gleichzeitig und asynchron über ein einziges Board betrieben werden, bei einer Abtastrate von bis zu 30 Pings pro Sekunde je Sensor.
- **Data quality:** Die Bibliothek bietet integrierte digitale Filter. Die Funktion `pingmedian` verarbeitet mehrere Messwerte zu einem Median, was Rauschen und fehlerhafte Echos effektiv aus dem Datensatz entfernt.

5.4.3. Funktionen

Die wichtigsten Befehle der NewPing-Bibliothek umfassen [Eck23]:

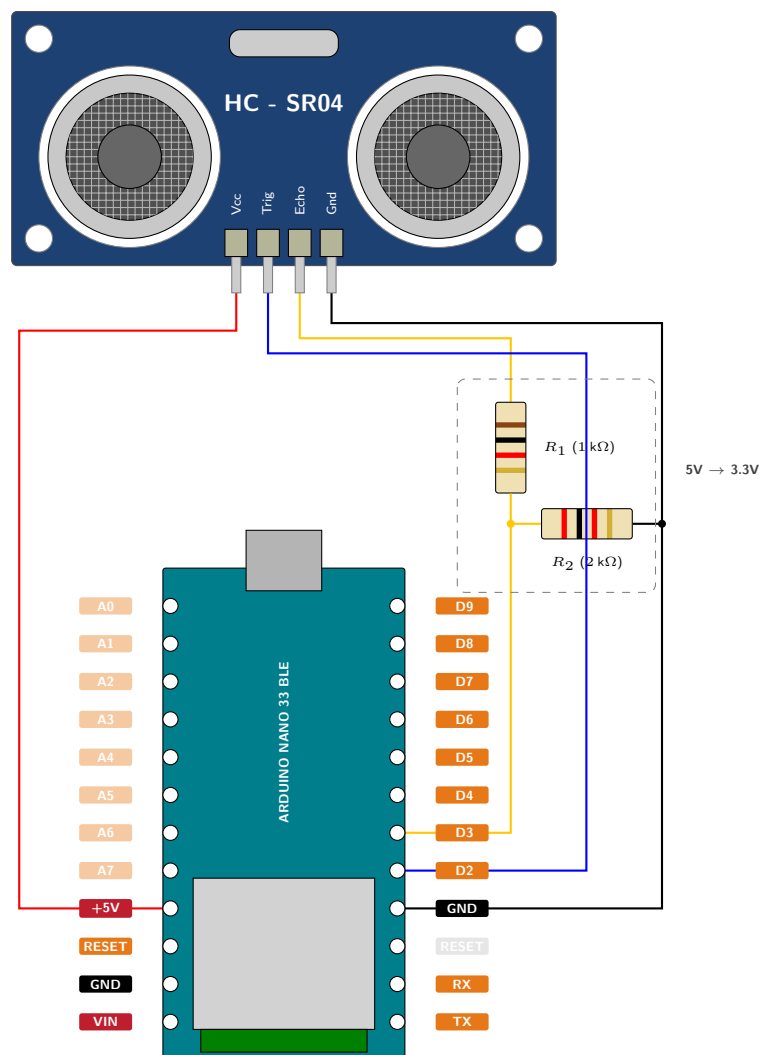


Abbildung 5.2.: Schaltplan Ultraschallsensor HC-SR04

- `NewPing sonar`(`TRIGGER_PIN`, `ECHO_PIN`, `MAX_DISTANCE` 5.1): Konstruktor zur Erstellung des Sensor-Objekts.
 - **TRIGGER_PIN (Der Auslöser)**: Dieser Anschluss entspricht Pin 2 auf dem HC-SR04 Modul und arbeitet mit einem regulären TTL-Pegel [KT-12]. Um einen neuen Messzyklus auszulösen, muss an diesem Triggereingang ein Signal mit einer fallenden Flanke für mindestens 10 μ s anliegen [KT-12]. Als direkte Reaktion auf dieses Signal sendet das Modul selbstständig ein 40 kHz Burst-Signal aus [KT-12]. In dem vorliegenden Code-Beispiel wird dieser Anschluss dem digitalen Pin 12 des Arduino zugewiesen.
 - **ECHO_PIN (Der Empfänger)**: Dieser Ausgang liegt auf Pin 3 des Sensormoduls und gibt das eigentliche Messergebnis ebenfalls als TTL-Pegel aus [KT-12]. Unmittelbar nach dem Aussenden des Ultraschall-Bursts springt dieser Pin auf einen HIGH-Pegel und wartet auf den Empfang des zurückkehrenden Echos [KT-12]. Sobald das reflektierte Signal detektiert wird, fällt der Ausgang sofort auf einen LOW-Pegel ab [KT-12]. Wird gar kein Echo erkannt (weil der Raum leer ist), verweilt der Pin für exakt 200 ms auf dem HIGH-Pegel, um dem Mikrocontroller die erfolglose Messung zu signalisieren [KT-12]. In der Software ist dieser Pin dem digitalen Pin 11 zugewiesen.
 - **MAX_DISTANCE (Die maximale Reichweite)**: Rein physikalisch besitzt der HC-SR04 eine messbare Distanz von 2 cm bis zu ca. 300 cm [KT-12]. Soll die maximale Distanz von 3 Metern erreicht werden, muss der Sensor sehr exakt auf das Objekt zielen, und es dürfen keine störenden Gegenstände im Sendekegel von 15° vorhanden sein [KT-12]. Im Programmcode ist `MAX_DISTANCE` eine festgelegte Konstante (hier auf 200 cm gesetzt), die dem `NewPing`-Objekt mitteilt, bis zu welcher Entfernung der Sensor überhaupt auf Objekte reagieren soll [Eck23].
- `sonar.ping()`: Sendet einen Ping und gibt die gemessene Laufzeit in Mikrosekunden (μ s) zurück.
- `sonar.ping_cm()`: Führt eine Messung durch und gibt die in Zentimeter (cm) umgerechnete Distanz zurück.
- `sonar.ping_median(int iterations)`: Führt die angegebene Anzahl an Pings durch, verwirft Ausreißer und gibt die mediane Laufzeit zurück.

5.5. Beispiel

In diesem Abschnitt wird ein einfaches, eigenständiges Codebeispiel beschrieben, welches die Funktionsweise und Integration der Bibliothek in C++ verdeutlicht.

5.5.1. Handbuch

1. Verbinden Sie VCC und GND des HC-SR04 mit 5V und GND des Boards [KT-12].
2. Verbinden Sie den TRIG-Pin mit Pin 12 und den ECHO-Pin mit Pin 11.
3. Öffnen Sie die Arduino IDE, kompilieren Sie den Code und laden Sie ihn hoch.
4. Öffnen Sie den Seriellen Monitor (Baudrate 115200). Hier werden nun die Messergebnisse ausgegeben[Ele20].

5.5.2. Funktion vom Beispiel

Das Skript bindet die `NewPing.h` ein. Über `#define` werden Pins und der maximale Messbereich (200 cm) definiert. In der Hauptschleife (`loop()`) stellt `delay(50)` sicher, dass zwischen zwei Pings gewartet wird, um Echosüberlappungen zu verhindern [Eck23]. Anschließend wird `ping_median(5)` aufgerufen. Ist ein Objekt vorhanden, muss die gemessene Zeit in eine Längeneinheit umgewandelt werden. Hierfür kommt der bibliotheksinterne Befehl `US_ROUNDTRIP_CM` zum Einsatz. Dies ist ein vordefinierter Teilerfaktor, mit dem Wert 57. Er basiert auf der physikalischen Schallgeschwindigkeit in der Luft. Da der Schall für einen Zentimeter Wegstrecke etwa 29 Mikrosekunden benötigt und das Signal den Weg doppelt zurücklegen muss (Hin- und Rückweg), entspricht 1 cm Objektabstand exakt einer Signallaufzeit von ca. 57 bis 58 Mikrosekunden. Durch die Division `uS / US_ROUNDTRIP_CM` wird die gemessene Zeit somit effizient und direkt in den korrekten Zentimeter-Abstand umgerechnet [Eck23].

5.5.3. Code

Listing 5.1.: Beispiel Code

```

1000 \begin{verbatim}
// File: Beispielcode.ino
1002 // Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
// to reduce noise.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-27
1006
#include <NewPing.h>
1008
// Digital pin for trigger signal
1010 #define TRIGGER_PIN 12
1012
// Digital pin for echo signal
#define ECHO_PIN 11
1014
// Maximum distance for the sensor (in cm)
1016 #define MAX_DISTANCE 200
1018
// Initialize NewPing object
NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1020
void setup() {
1022   Serial.begin(115200);
   Serial.println("System gestartet. Beginne Messung...");
1024 }

1026 void loop() {
// Wait between pings (min. 29ms recommended)
1028   delay(50);

1030   // Get median of 5 measurements (in microseconds)
   unsigned int uS = sonar.ping_median(5);
1032
   Serial.print("Gefilterte Distanz: ");
1034
   if (uS == 0) {
1036     Serial.println("Außerhalb der Reichweite");
   } else {
1038     // Convert microseconds to centimeters
     Serial.print(uS / US_ROUNDTRIP_CM);
1040     Serial.println(" cm");
   }
1042 }
\end{verbatim}

```

../Code/Ultraschallsensor/Beispielcode/Beispielcode.ino

5.6. Kalibrierung

Die Ultraschallmessung beruht auf der Ausbreitungsgeschwindigkeit von Schall in der Luft. Um das System zu überprüfen wird ein Objekt in einem exakt gemessenem Abstand vor dem Sensor aufgestellt und mit der Ausgabe im seriellen Monitor verglichen. Die Standardberechnung der Bibliothek geht von einer konstanten Schallgeschwindigkeit von $c \approx 343 \text{ m/s}$ bei 20°C aus [Fra16]. Da die Schallgeschwindigkeit temperaturabhängig ist, ändert sie sich um ca. $0,6 \text{ m/s}$ pro Grad Celsius [Fra16]. Für hochpräzise Anwendungen muss das System kalibriert werden (z.B. über einen DHT22-Sensor). Die dynamische Berechnung lautet:

$$\text{Distanz} = \frac{\text{Laufzeit}}{2} \times (331,3 + 0,606 \times \text{Temperatur in } ^\circ\text{C})$$

5.7. Einfacher Code ohne newPing Bibliothek

Der folgende Code demonstriert eine Abstandsmessung ohne Einbindung der NewPing-Bibliothek und nutzt `pulseIn()`.

Listing 5.2.: Beispiel Code ohne Bibliothekseinbindung

```

1000 \begin{verbatim}
// File: Beispielcode2.ino
1002 // Standalone script for filtered distance measurement with HC-SR04.
// Uses NewPing library to get a median value from multiple measurements
// to reduce noise.
1004 // Author: Wilko Hinrichs
// Date: 2026-04-27
1006
const int TRIG_PIN = 12; // Pin for trigger signal
1008 const int ECHO_PIN = 11; // Pin for echo signal
1010
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  Serial.begin(115200);
1014 }
1016
void loop() {
  digitalWrite(TRIG_PIN, LOW);
  delayMicroseconds(2);
1018
  // 10 microseconds HIGH pulse (ends with falling edge)
  digitalWrite(TRIG_PIN, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);
1024
  // pulseIn measures the length of the ECHO pulse
  long duration = pulseIn(ECHO_PIN, HIGH);
1026
  // Calculate distance (travel time in us / 58)
  float distance = duration / 58.0;
1030
  Serial.print("Gemessene Distanz: ");
  Serial.print(distance);
  Serial.println(" cm");
1032

```

```

1034 // Keep interval (> 20ms)
1036 delay(50);
    }
1038 \end{verbatim}

```

../Code/Ultraschallsensor/Beispielcode2/Beispielcode2.ino

5.8. Einfache Anwendung

Eine anschauliche Anwendung für den HC-SR04 ist eine automatisierte Einparkhilfe. Fährt ein Fahrzeug heran, liest der Sensor die Distanz. Unterschreitet der distance-Wert 50 cm, schaltet das System eine Warn-LED ein. Fällt der Wert auf unter 10 cm, wird ein Buzzer getriggert [Eck23].

Listing 5.3.: Einparkhilfe Beispielcode

```

1000 \begin{verbatim}
    // File: EinparkhilfeCode.ino
1002 // Application example: Automated parking assistant.
    // System shows distance via LED and warns acoustically on critical
    // approach.
1004 // Author: Wilko Hinrichs
    // Date: 2026-04-28
1006
    #include <NewPing.h>
1008
    // Pin configuration
1010 #define TRIGGER_PIN 12
    #define ECHO_PIN 11
1012 #define MAX_DISTANCE 200 // Maximum scan range in cm
    #define LED_PIN 3 // Yellow warning LED
1014 #define BUZZER_PIN 4 // Acoustic buzzer
1016
    // Initialize sensor
    NewPing sonar(TRIGGER_PIN, ECHO_PIN, MAX_DISTANCE);
1018
    void setup() {
1020     Serial.begin(115200);
        pinMode(LED_PIN, OUTPUT);
1022     pinMode(BUZZER_PIN, OUTPUT);
    }
1024
    void loop() {
1026     // Keep measurement interval
        delay(50);
1028
        // Measure distance in cm
1030     unsigned int distance = sonar.ping_cm();
1032
        Serial.print("Abstand: ");
        Serial.print(distance);
1034     Serial.println(" cm");
1036
        // Logic check based on distance thresholds
        if (distance > 0 && distance < 10) {
1038             // Critical area: LED and buzzer active
            digitalWrite(LED_PIN, HIGH);
1040             digitalWrite(BUZZER_PIN, HIGH);
        }
1042     else if (distance >= 10 && distance < 50) {
        // Warning area: Only LED active
1044         digitalWrite(LED_PIN, HIGH);
            digitalWrite(BUZZER_PIN, LOW);
    }
    }
\end{verbatim}

```

```

1046 }
1047 else {
1048   // Safe area: All off
1049   digitalWrite(LED_PIN, LOW);
1050   digitalWrite(BUZZER_PIN, LOW);
1051 }
1052 }
\end{verbatim}

```

../Code/Ultraschallsensor/EinparkhilfeCode/EinparkhilfeCode.ino

5.9. Tests

5.9.1. Einfacher Funktionstest

Für die Überprüfung der grundlegenden Schaltung und Sensorik wird das Standalone-Skript aus Abschnitt 5.1 (Gefilterte Distanzmessung mit der NewPing-Bibliothek) auf den Mikrocontroller geladen.

Ein harter, planer Gegenstand (z. B. ein Buch) wird in exakt 15 cm Entfernung parallel zur Sensorfront platziert. Anschließend wird der Serielle Monitor der Arduino IDE mit einer Baudrate von 115200 geöffnet. Der ausgegebene Wert muss durch den Aufruf der Funktion `sonar.ping_median(5)` sehr stabil bei 15 cm liegen. Die Schwankung darf dabei die gerätespezifische Auflösung des Moduls von 3 mm [KT-12] kaum überschreiten. Das Skript bestätigt somit, dass das Sende- und Empfangsmodul korrekt arbeiten und die serielle Datenübertragung fehlerfrei funktioniert.

5.9.2. Alle Funktionen testen

Ein vollständiger Systemtest kann wie folgt durchgeführt werden. Hierfür können die Ausgaben des Seriellen Monitors aus dem vorherigen Testcode 5.1 oder das Verhalten der LEDs aus dem Parkassistenten-Code 5.3 beobachtet werden:

- **Minimaldistanz:** Das Objekt wird auf unter 2 cm an den Sensor herangeführt. Da der Sensor nach dem Senden des 200 µs langen Bursts eine kurze Umschaltzeit benötigt [KT-12], kann er Echos im absoluten Nahbereich physikalisch nicht mehr auflösen. Der Monitor sollte hier durch die Bibliothek korrekterweise Außerhalb der Reichweite (Rückgabewert 0) ausgeben [Eck23].
- **Maximaldistanz:** Der Sensor wird in einen leeren, großen Raum gerichtet, sodass das nächste Hindernis mehr als 300 cm entfernt ist. Laut Spezifikation verweilt der ECHO-Pin bei einem fehlenden Echo für 200 ms auf einem HIGH-Pegel [KT-12]. Der Test belegt, dass die NewPing-Bibliothek diesen Hardware-Timeout sicher abfängt, ohne dass sich der Mikrocontroller in einer blockierenden Endlosschleife aufhängt [Eck23].
- **Winkeltest (Abstrahlkegel):** Ein Objekt wird in 50 cm Entfernung positioniert und langsam seitlich aus dem Zentrum bewegt. Überschreitet der Winkel 15° zur Mittelachse, wird die Fläche vom Objekt für die Ultraschallwellen zu klein [KT-12]. Die Messwerte brechen ab, was die seitlichen Grenzen des Erfassungsbereichs bestätigt.
- **Dynamischer Reaktionstest:** Mit dem Code des automatisierten Parkassistenten aus Abschnitt 5.3 wird die Reaktionsgeschwindigkeit der Gesamtanlage getestet. Ein Objekt wird zügig auf den Sensor zubewegt. Die gelbe Warn-LED muss exakt beim Unterschreiten der 50-cm-Marke aufleuchten, und der Buzzer muss ohne spürbare Software-Verzögerung bei exakt 10 cm triggern.

5.10. Weiterführende Literatur

SparkFun Electronics / ElecFreaks (2019): HC-SR04 Ultrasonic Sensor Datasheet, Verfügbar unter: <https://www.sparkfun.com/datasheets/Sensors/Ultrasonic/HCSR04.pdf> [abgerufen am 28.04.2026]

- Diese technische Spezifikation bildet die fundamentale Grundlage für die Ansteuerung. Sie enthält die exakten Timing-Diagramme für das Trigger- und Echo-Signal sowie Angaben zum Öffnungswinkel des Schallkegels, die für die Gehäuseplanung wichtig sind.

Eckel, Tim (2023): Arduino NewPing Library Documentation, GitHub/Bitbucket Repository, Verfügbar unter: <https://bitbucket.org/teckel12/arduino-newping/wiki/Home> [abgerufen am 28.04.2026]

- Eine umfassende Dokumentation zur verwendeten Programmbibliothek NewPing Library von Tim Eckel. Es werden die Vorteile gegenüber herkömmlichen Messmethoden erläutert und die Implementierung von Filtern wie der `ping_median()`-Funktion zur Rauschunterdrückung detailliert beschrieben.

Ekbert Hering und Gert Schönfelder, Sensoren in Wissenschaft und Technik Funktionsweise und Einsatzgebiete, Springer Vieweg, 2023, ISBN: 9783658394905 [HS23]

- Eine umfassende und aktuelle Einführung in die Grundlagen und Anwendungen von Sensoren in Technik, Biologie und Medizin. Das Buch behandelt neben physikalischen Grundlagen auch Themen wie Schnittstellen, Sicherheit und Signalverarbeitung bei Ultraschallwandlern.

6. SG90 Servomotor

Einleitung

Dieses Kapitel beschreibt den mechanischen Aktor des Systems, den Micro-Servomotor TowerPro SG90. Es werden die technischen Spezifikationen, die hardwareseitige Einbindung sowie die softwareseitige Ansteuerung über die Standard-Bibliotheken detailliert erläutert.

6.1. Allgemeines

Ein Servomotor ist ein geschlossenes mechatronisches System (Closed-Loop), das für präzise Winkelpositionierungen eingesetzt wird. Im Gegensatz zu kontinuierlich drehenden Gleichstrommotoren besteht ein Standard-Servo aus einem DC-Motor, einem Untersetzungsgetriebe, einem Potentiometer zur Positionserfassung und einer Steuerelektronik. Der Mikrocontroller sendet ein Pulsweitenmodulations-Signal (*PWM*), welches die Steuerelektronik auswertet und den Motor exakt auf den gewünschten Winkel dreht und dort aktiv hält. Servomotoren findet man vor allem im RC-Modellbau (z.B. Lenkung, Ruder, Klappen), in Robotergelenken sowie in Schwenkmechanismen für Kameras oder Sensoren, bei denen wiederholgenaue und stabile Positionen erforderlich sind.[Mik20]

6.2. Servomotor TowerPro SG90

Der TowerPro SG90 ist einer der weltweit am häufigsten verwendeten Micro-Servomotoren für kleine Projekte. Er zeichnet sich durch sein extrem geringes Gewicht, seine kompakte Bauform und sein Nylon-Zahnradgetriebe aus. Der Aktor verfügt über ein dreipoliges Anschlusskabel: Braun für die Masse (*GND*), Rot für die Versorgungsspannung (*VCC*) und Orange für das PWM-Steuersignal (*Signal*). Der TowerPro SG90 hat einen mechanischen Rotationsspielraum von 180 Grad (0° bis 180°). Gesteuert wird er über ein 50-Hz-PWM-Signal (20 ms Periodendauer). Eine Pulsweite von 1 ms entspricht theoretisch 0°, 1,5 ms entsprechen 90° (Mittelstellung) und 2 ms entsprechen 180°.[Pat; Com17]

6.3. Spezifikation

- **Quellen referenzieren:** Die physikalischen und elektrischen Kennwerte beziehen sich auf technische Angaben zum SG90 von Elektronik-Kompendium und Components101.[Pat; Com17]
- **Extrahierte Informationen:**
 - Betriebsspannung: 4,8 V (ca. 5 V)
 - Stellmoment (Stall Torque): 1,8 kg/cm (bei 4,8 V)
 - Stellgeschwindigkeit (ohne Last): 0,1 Sekunden / 60 Grad (bei 4,8 V)
 - Gewicht: 9 Gramm
 - Getriebetyp: Kunststoff (Nylon)

- Steuersignal: PWM (50 Hz / 20 ms Periode)
- Dimensionen: ca. 22,2 x 11,8 x 31 mm

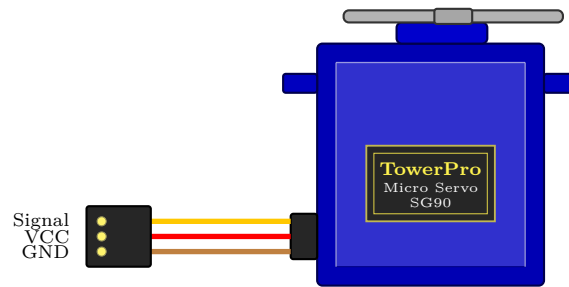


Abbildung 6.1.: Schematische Darstellung des Servomotors TowerPro SG90

- **Schaltplan:** Das rote Kabel (*VCC*) des SG90 wird mit dem 5V-Ausgang (*VUSB*) des Systems verbunden, um ausreichend Leistung für den Motor bereitzustellen. Das braune Kabel (*GND*) wird mit der gemeinsamen Masse verbunden. Das orange Kabel (*Signal*) wird an einen digitalen PWM-fähigen Pin (z.B. Pin 9) des Mikrocontrollers angeschlossen.[Pat]

Hinweis: Der Arduino Nano 33 BLE Sense arbeitet mit 3,3V-I/O-Pegeln und ist nicht 5V-tolerant.[Ard26a] Viele Servomotoren können jedoch ein 3,3V-PWM-Steuersignal verarbeiten, sofern der Servo selbst mit einer geeigneten Versorgungsspannung betrieben wird und eine gemeinsame Masse vorhanden ist.[Dr]

6.4. Bibliothek

6.4.1. Beschreibung

Für die Ansteuerung des SG90 wird in der Arduino-Umgebung die integrierte *Servo*-Bibliothek genutzt[Lib25a]. Dadurch entfällt für den Programmierer die aufwändige Erzeugung des exakten 50-Hz-PWM-Signals, und der Motor kann bequem über die direkte Vorgabe eines Winkelwertes von 0 bis 180 Grad angesteuert werden.[Lib25a]

6.4.2. Installation

- **Installation:** Die Bibliothek kann in der Arduino IDE über den Bibliotheksverwalter eingebunden werden. Im Programm wird sie anschließend mit `#include <Servo.h>` importiert.[Mik20]
- **Data types:** Die Zielwinkel werden als `int` (Integer/Ganzzahlen) übergeben. Für präzisere Steuerungen können Pulsweiten in Mikrosekunden ebenfalls als `int` übergeben werden (vgl. [Lib25b]).
- **Data structure:** Jeder Servomotor wird im Code als Objekt der Klasse *Servo* deklariert (z.B. `meinServo;`).
- **Data quantity:** Auf Standard Arduino-Boards unterstützt die Bibliothek bis zu 12 Servos gleichzeitig; auf speziellen Architekturen (wie der Mbed-OS Architektur des Nano 33 BLE) kann dies variieren, reicht jedoch für mehrere Aktoren vollkommen aus.[Doc26]
- **Data quality:** Die Bibliothek arbeitet timer- und interruptbasiert. Die Servosignale werden im Hintergrund mit dem zuletzt über `write()` gesetzten Wert erzeugt, wodurch keine manuelle PWM-Erzeugung mit blockierenden Delay-Schleifen notwendig ist.[Lib25b]

6.4.3. Funktionen

Die wichtigsten Methoden der Bibliothek sind:

- `attach(pin)`: Verbindet das Servo-Objekt mit einem bestimmten digitalen Pin.[Ard26b]
- `write(angle)`: Dreht den Servo auf den entsprechenden Winkel (0° bis 180°).
- `writeMicroseconds(uS)`: Ermöglicht die Ansteuerung über Pulsweiten in Mikrosekunden.[Ard26d]
- `read()`: Gibt den zuletzt gesetzten Servo-Winkel zurück.[Ard26c]

6.5. Beispiel

Dieser Abschnitt zeigt ein elementares Beispiel „Sweep“, bei dem der Servo kontinuierlich von 0 auf 180 Grad und wieder zurück schwenkt.

6.5.1. Handbuch

Schließen Sie das rote Kabel an 5V, das braune an GND und das orange Signalkabel an Pin 9 des Arduinos an. Kompilieren Sie das Programm und laden Sie es auf das Board. Der Arm des Servomotors sollte nun anfangen, langsam und gleichmässig hin und her zu wischen.

6.5.2. An einem Beispiel

Das Skript in 6.1 bindet zunächst die Bibliothek `Servo.h` ein und erstellt mit `radarServo` ein Servo-Objekt. Zusätzlich wird die Variable `pos` zur Speicherung der aktuellen Winkelposition verwendet. In der `setup()`-Routine wird der Motor über `radarServo.attach(9)` an den digitalen Pin 9 gebunden.

In der `loop()`-Schleife sorgen zwei `for`-Schleifen für die kontinuierliche Schwenkbewegung des Servomotors. Die erste Schleife erhöht den Winkel schrittweise von 0° auf 180°. Dabei wird mit `radarServo.write(pos)` jeweils eine absolute Zielposition angefahren. Nach jeder Winkeländerung folgt mit `delay(15)` eine kurze Pause von 15 ms, damit der Motor die physikalische Position erreichen kann. Die zweite Schleife führt anschließend die Rückwärtsbewegung von 180° zurück auf 0° aus. Dadurch entsteht eine fortlaufende Hin- und Herbewegung des Servomotors.

6.5.3. Code

Listing 6.1.: Beispiel Code für den TowerPro SG90

```

1000 /**
      * @file      Servomotor.ino
1002  * @author    Simon Müller
      * @date      2026-05-01
1004  * @version   1.0
      *
1006  * @brief     Servo-based radar scanning system.
      *
1008  * This sketch controls a TowerPro SG90 (or similar) servo to perform
      * a continuous back-and-forth sweep from 0 deg to 180 deg and back.
1010  * All position commands are absolute angles
      * (0180 deg range).
1012  */
1014 #include <Servo.h>

```

```

1016 Servo radarServo;    ///< Servo object used for radar scanning
1017 int pos = 0;          ///< Current servo position in degrees (0180,
1018                        absolute)
1019
1020 /**
1021  * @brief Initializes the servo on startup.
1022  *
1023  * Attaches the servo object to digital pin 9 and prepares it
1024  * for subsequent position commands. The servo will interpret
1025  * all following write() calls as absolute target angles in
1026  * the range 0°180°.
1027  */
1028 void setup() {
1029     radarServo.attach(9); // Attach servo to pin 9
1030 }
1031
1032 /**
1033  * @brief Main loop for continuous servo motion.
1034  *
1035  * The servo is moved in two phases using absolute angle commands:
1036  * - Phase 1: Sweep from 0° to 180° in 1° steps
1037  * - Phase 2: Sweep from 180° back to 0° in 1° steps
1038  *
1039  * Each call to radarServo::write(pos) sets an absolute target
1040  * angle; it does NOT move the servo by a relative offset.
1041  * With a 15 ms delay between steps, one complete cycle
1042  * takes approximately 5.4 seconds.
1043  */
1044 void loop() {
1045     // Phase 1: sweep from 0° to 180° (absolute positions)
1046     for (pos = 0; pos <= 180; pos += 1) {
1047         radarServo.write(pos); ///< Set servo to absolute angle 'pos'
1048         delay(15);             ///< Wait 15 ms to reach the position
1049     }
1050
1051     // Phase 2: sweep from 180° back to 0° (absolute positions)
1052     for (pos = 180; pos >= 0; pos -= 1) {
1053         radarServo.write(pos); ///< Set servo to absolute angle 'pos'
1054         delay(15);             ///< Wait 15 ms
1055     }
1056 }

```

../Code/Servomotor/Servomotor.ino

6.5.4. Dateien

Für das Ausführen des Codes wird ausschließlich die Standard-Headerdatei `Servo.h` benötigt, welche in der Arduino IDE vorinstalliert ist.

6.6. Kalibrierung

Bei Servomotoren können die praktisch nutzbaren Endpositionen von den theoretischen PWM-Werten abweichen. Wird der Motor mit zu kleinen oder zu großen Pulsweiten an seine mechanischen Grenzen gefahren, kann dies zu Brummen, Vibrationen oder mechanischer Belastung führen[Ard26d]. Zur Kalibrierung bietet die `attach()`-Funktion optionale Parameter für die minimalen und maximalen Mikrosekunden: `radarServo.attach(9, 500, 2400)`. Diese Werte können durch iteratives Ausprobieren mit `writeMicroseconds()` an den individuellen Motor angepasst werden, bis der gewünschte Winkelbereich ohne mechanisches Blockieren erreicht ist.

6.7. Einfacher Code (ohne Bibliothek)

Der Code in 6.2 erzeugt das PWM-Signal für den Servo manuell ohne die Verwendung der Servo-Bibliothek. In der `setup()`-Routine wird Pin 9 mit `pinMode(9, OUTPUT)` als Ausgang konfiguriert. Die `loop()`-Funktion generiert anschließend ein 50-Hz-Signal. Dazu wird der Pin mit `digitalWrite(9, HIGH)` für 1500 µs auf HIGH gesetzt, was ungefähr einer Servo-Position von 90° entspricht. Danach wird der Pin mit `digitalWrite(9, LOW)` für 18500 µs auf LOW gesetzt, sodass eine vollständige Periodendauer von 20 ms entsteht.

Diese Methode zeigt vereinfacht, wie die Ansteuerung eines Servomotors über ein PWM-Signal funktioniert. Da der Ablauf jedoch durch die verwendeten Verzögerungen blockierend ist, eignet sich diese Variante nur eingeschränkt für komplexere Anwendungen. In solchen Fällen sollte die Servo-Bibliothek verwendet werden, da diese mit Timern arbeitet und die Ansteuerung komfortabler übernimmt.

Listing 6.2.: Code ohne Bibliothek für den TowerPro SG90

```

1000 /**
1001  * @file ServoManual.ino
1002  * @brief Manual servo control without library
1003  *
1004  * @details Demonstrates servo motor control through direct PWM
1005  *          generation
1006  * using digitalWrite() and delayMicroseconds(). The servo is positioned
1007  * at approximately 90° (center position). This method shows how the
1008  * Servo library works at the hardware level.
1009  *
1010  * @author Simon Müller
1011  * @date 01.05.2026
1012  * @version 1.0
1013  *
1014  * @section hardware Hardware
1015  * — Arduino Board (e.g. Uno, Nano)
1016  * — Servo motor connected to Pin 9
1017  * — Power supply: 5V for servo (VCC and GND)
1018  *
1019  * @section notes Notes
1020  * This implementation uses blocking delays and is for educational
1021  * purposes.
1022  * For production code, use the Arduino Servo library instead.
1023  */
1024
1025 int servoPin = 9; ///< Pin number for servo control signal
1026
1027 /**
1028  * @brief Initialization at program startup
1029  *
1030  * @details Configures the servo pin as OUTPUT to enable
1031  * sending PWM signals to the servo motor.
1032  *
1033  * @return void
1034  */
1035 void setup() {
1036     pinMode(servoPin, OUTPUT); // Set servo pin as output
1037 }
1038
1039 /**
1040  * @brief Main loop for continuous PWM generation
1041  *
1042  * @details Manually generates a 50 Hz PWM signal (20 ms period).
1043  * The HIGH pulse of 1.5 ms corresponds to a servo position of approx.
1044  * 90°.
1045  *
1046  * PWM Timing:
1047  * — HIGH phase: 1500 µs (1.5 ms) 90° position (center)

```

```

1046 * - LOW phase: 18500  $\mu$ s (18.5 ms) fills up to 20 ms total period
1047 * - Frequency: 50 Hz (1000000  $\mu$ s / 20000  $\mu$ s = 50 Hz)
1048 *
1049 * @note This method blocks program execution. For more complex
1050 * applications, the Servo library should be used.
1051 *
1052 * @return void
1053 */
1054 void loop() {
1055     // Move to approx. 90 degrees (1.5ms HIGH pulse)
1056     digitalWrite(servoPin, HIGH);    ///< Starts the HIGH pulse
1057     delayMicroseconds(1500);          ///< Pulse width: 1.5 ms 90°
1058     position
1059     digitalWrite(servoPin, LOW);      ///< Ends the HIGH pulse
1060
1061     // Fill the remaining 20ms period (50 Hz)
1062     delayMicroseconds(18500);         ///< LOW phase: 18.5 ms until next
1063     period
1064 }

```

../Code/Servomotor/ServomotorohneBib.ino

6.8. Einfache Anwendung

Eine klassische Anwendung für den SG90 ist die Basis eines **Ultraschall-Radarsystems**. Der Ultraschallsensor HC-SR04 wird auf der Achse des Servos montiert. Der Servo tastet systematisch in 1-Grad-Schritten die Umgebung vor sich ab (0° bis 180°). Bei jedem Schritt wird eine Entfernungsmessung durchgeführt. Diese Kombination aus Aktor-Winkel und Sensor-Distanz ermöglicht es der Software (z. B. in Processing), eine zweidimensionale „Radarkante“ der Umgebung auf dem Bildschirm zu zeichnen.[die25]

6.9. Tests

6.9.1. Einfacher Funktionstest

Mit dem „Sweep“-Code aus Abschnitt 6.2 lässt sich prüfen, ob der Motor korrekt verkabelt ist. Der Motor sollte möglichst ruckelfrei und gleichmässig schwenken. Ein stotternder Motor oder ein Neustart des Arduinos während der Bewegung kann auf eine unzureichende Stromversorgung, beispielsweise einen VCC-Abfall, hinweisen.

6.9.2. Alle Funktionen testen

Ein vollständiger Systemtest umfasst:

1. **Grenzttest:** Anfahren von 0° und 180°. Der Motor darf in diesen Endlagen nicht surren oder ungewöhnlich warm werden.[Ard26d]
2. **Geschwindigkeitstest:** Befehle `write(0)` und sofort danach `write(180)` ohne Delay. Messung, ob der Motor die physikalische Herstellerangabe von 0,3 Sekunden für diesen 180°-Weg einhalten kann.[Com17]
3. **Präzisionstest:** Verwendung von `writeMicroseconds(1500)` zur exakten Zentrierung, gefolgt von der Überprüfung der Winkelgenauigkeit mittels eines Geodreiecks auf dem Rotorkopf.[Ard26d]

6.10. Weiterführende Literatur

Teil III.

Domain Knowledge - Tools

7. Python Tool XY

8. Hardware Tool XY

Teil IV.

Development

9. Flow Chart Development XY

Teil V.

Monitoring

10. Monitoring XY

Teil VI.

**Verification - Evaluation -
Conclusion**

11. Evaluation/Verification XY

12. To DoX Y

13. Conclusion XY

Teil VII.

Anhang

14. Bill Of Material

Tabelle 14.1.: Hardware Bill of Materials

Bild	Stückzahl	Beschreibung	Zulieferer	Preis()
	1	Arduino Nano 33 BLE Sense	reichelt.de	33.03
	1	Ultraschallsensor HC-SR-04	reichelt.de	2.90
	1	TowerPro Servomotor SG90	az-delivery.de	5.99
	1	Breadbord 25 Kontakte	reichelt.de	2.39
	1	LED grün 8mm	reichelt.de	0.19
	1	LED rot 8mm	reichelt.de	0.19
	1	Entwicklerboards - Steckbrückenkabel 40 Pole	reichelt.de	1.55
	1	Micro-USB Kabel	reichelt.de	5.26
	2	220 Ohm Widerstand	reichelt.de	0.18
	1	2k Ohm Widerstand	reichelt.de	0.10
	1	1k Ohm Widerstand	reichelt.de	0.09
Total				≈ 51,87

15. **SBOM**

`requirements.txt`

16. Package Example

16.1. Introduction

16.2. Description

16.3. Installation

`pip packageExample`

16.4. Example - Manual

16.5. Example

16.6. Example - Code

16.7. Example - Files

`PackageExample.py`

16.8. Further Reading

Literatur

- [Aba+16] M. Abadi u. a. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.

Teil VIII.

**Hints - Examples for LaTeX -
Read, Use and Remove**

17. Hinweise

Bitte verwenden Sie **biber.exe**.

Wenn Sie ein Symbolverzeichnis erstellen möchten, rufen Sie makeindex auf:
makeindex %.nlo -s nomencl.ist

17.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier-Kurven werden mit Hilfe von Bernsteinpolynomen gebildet. Wenn mindestens zwei Punkte sowie zwei Tangenten bekannt sind, können Kontrollpunkte ermittelt werden mit denen ein Kontrollpolygon gebildet wird. An diesem Kontrollpolygon orientiert sich der Verlauf der Kurve. Die Anzahl der Kontrollpunkte ist abhängig vom Grad der Bézier-Kurve. Eine Bézier-Kurve vom Grad n hat $n+1$ Kontrollpunkte. Die Berechnung der Bézier-Kurve erfolgt über den De-Casteljau-Algorithmus.

Definition. Im Intervall $[0; 1]$ ist das **Bernstein-Polynom n -ten Grades** definiert durch:

$$b_{i,n}(\lambda) = \binom{n}{i} (1-\lambda)^{n-i} \lambda^i, \quad \lambda \in [0, 1], \quad i = 0, \dots, n \quad (17.1)$$

Definition. Der Binomialkoeffizient ist definiert durch:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \dots, n \quad (17.2)$$

Hierbei bilden die Bernstein-Polynome $b_{i,n}$ eine Basis des Vektorraumes $\mathbb{P}^n(I)$ der Polynome höchstens vom Grad n über I . Somit lässt sich jedes Polynom höchstens n -ten Grades eindeutig als Linearkombination schreiben. [Far02]

Beispiel. Bestimmung von Bernstein-Polynome für $n = 3$ gilt:

$$\begin{aligned} b_{3,0}(\lambda) &= \binom{3}{0} (1-\lambda)^{3-0} \lambda^0 = (1-\lambda)^3 \\ b_{3,1}(\lambda) &= \binom{3}{1} (1-\lambda)^{3-1} \lambda^1 = 3\lambda(1-\lambda)^2 \\ b_{3,2}(\lambda) &= \binom{3}{2} (1-\lambda)^{3-2} \lambda^2 = 3\lambda^2(1-\lambda) \\ b_{3,3}(\lambda) &= \binom{3}{3} (1-\lambda)^{3-3} \lambda^3 = \lambda^3 \end{aligned}$$

$b_{3,i}(\lambda)$ mit $i = 0, 1, 2, 3$ sind die kubischen Bernstein-Polynome von Grad 3.

Definition. Gegeben seien die Punkte

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \dots, n$$

Eine **Bézier-Kurve** ist dann definiert durch

$$C(\lambda) = \sum_{i=0}^n Q_i \cdot b_{i,n}(\lambda) \quad (17.3)$$

Die Punkte $Q_i, i = 0, \dots, n$ heißen **Kontrollpunkte**.

Die Kontrollpunkte einer Bézier-Kurve bilden das sogenannte Kontrollpolygon.

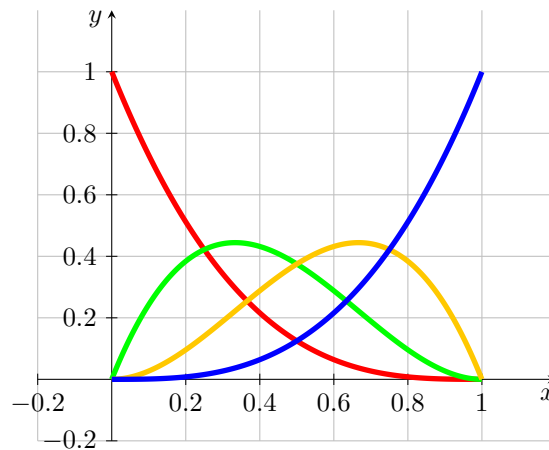


Abbildung 17.1.: Bernstein-Polynom vom Grad 3

Bemerkung. Es sei der Startpunkt P_0 und der Endpunkt P_1 , sowie die Tangenten $\vec{t}_0$ und $\vec{t}_1$ gegeben. Die Tangenten sind nicht notwendigerweise normiert. Die Kontrollpunkte Q_0, Q_1, Q_2 und Q_3 der zugehörigen Bézier-Kurve sind durch folgende Gleichungen gegeben:

$$Q_0 = P_0, \quad Q_1 = P_0 + \lambda_0 \vec{t}_0, \quad Q_2 = P_1 - \lambda_1 \vec{t}_n, \quad Q_3 = P_1 \quad (17.4)$$

[Jak+10]

Beispiel. Gegeben seien

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Dann ergibt sich gemäß Satz 17.4 für Kontrollpunkte der zugehörigen Bézier-Kurve:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \quad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

Die Bézier-Kurve und ihre Kontrollpunkte sind in der Abbildung 17.2 dargestellt.

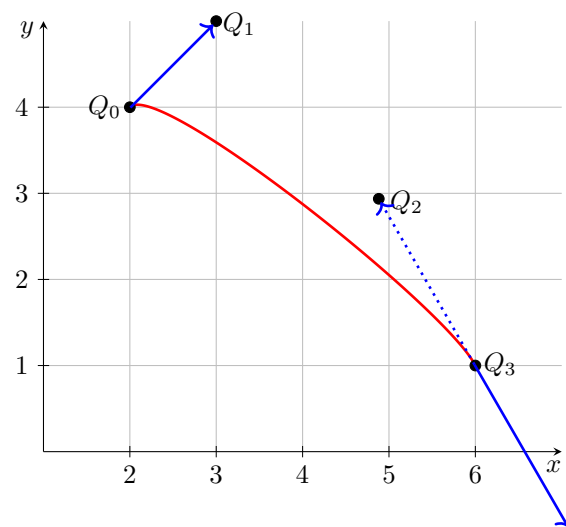


Abbildung 17.2.: Bézier-Kurve zum Beispiel 17.1

18. Verrundung mit einem Kreisbogen

18.1. Gleichungen

Die Verrundung mit einem Kreisbogen ist eine einfache Variante der Glättung von Ecken. Diese Variante ermöglicht die Bearbeitung und die Erzeugung von Dateien gemäss DIN 66025. Zur Glättung werden stets drei Punkte P_0 und S und P_1 betrachtet, damit ergibt sich ein symmetrisches Hermite-Problem. Gemäss Satz ?? wird vorausgesetzt, dass es sich in der Standardform $HP(L, \alpha)$ befindet.

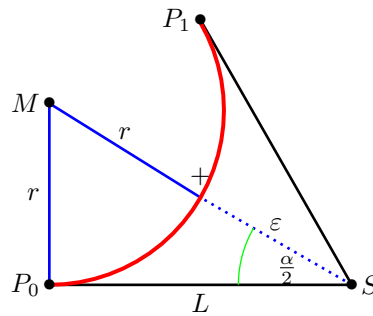


Abbildung 18.1.: Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck

Die Abbildung 18.1 stellt die Situation dar. Die Punkte P_0 , S und P_1 sind gegeben. Der eingeschlossene Winkel $\alpha = \angle(P_0; S; P_1)$ sowie der Abstand $L = \|S - P_0\| = \|P_1 - S\|$ sind in der Grafik dargestellt. Der rote Kreisbogen ist das gewünschte Ergebnis. Der Abstand seines Mittelpunkts M zu S beträgt dann $r + \varepsilon$, wobei r der Radius des Kreisbogens und ε die vorgegebene Toleranz ist. Somit erhalten wir ein rechtwinkliges Dreieck P_0SM , dessen Kantenlängen L , $r + \varepsilon$ und r sind.

Gemäss der Definition des Sinus und des Tangens folgt:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r + \varepsilon} \quad \text{und} \quad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \quad \text{und} \quad r = \cot\left(\frac{\alpha}{2}\right) L$$

Die beiden Gleichung können zusammengefasst werden, so dass sich folgende Bedingungen ergeben:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \quad \text{bzw.} \quad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

Damit ergibt sich für den Radius die Gleichung:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha)}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

Der Faktor zum Umrechnung von L und ε kann mit Hilfe trigonometrischer Umformungen vereinfacht werden.

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1 - \sqrt{\frac{1 - \cos(\alpha)}{2}}}{\sqrt{\frac{1 + \cos(\alpha)}{2}}} = \frac{\sqrt{2} - \sqrt{1 - \cos(\alpha)}}{\sqrt{1 + \cos(\alpha)}} = \frac{\sqrt{1 + \cos(\alpha)} - \sin(\alpha)}{1 + \cos(\alpha)}$$

Durch den Vergleich mit dem symmetrischen Hermite-Problem in Standardform ergibt sich dann die folgende Notation:

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}}$$

Die vorherigen Überlegungen werden nun im nachfolgenden Satz zusammengefasst.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

Dann kann ein Kreisbogen gefunden werden, der die Punkte P_0 und P_1 verbindet, die gleichen Tangentenrichtungen in den beiden Punkten besitzt.

Für die Kreisbogen gilt:

$$r = L \cdot \left| \frac{P_{1,x}}{P_{1,y}} \right|; \quad \phi_0 = -\text{sign}(\alpha) \cdot \frac{\pi}{2}; \quad \phi_1 - \phi_0 = \text{sign}(\alpha) \cdot \pi - \alpha = \beta;$$

$$M = P_0 + \text{sign}(\alpha) \cdot r \cdot \vec{t}_0^\perp = \text{sign}(\alpha) \cdot r \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Aus der Vorgabe des Abstand L kann, wie in Satz 18.1 dargestellt, der maximale Fehler berechnet werden. Gemäss der Herleitung ist es auch möglich, den maximalen Fehler ε vorzugeben und daraus den maximalen Abstand L zu bestimmen.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

- a) Es sei der maximale Fehler ε vorgegeben. Der maximale Abstand L , für den ein Kreisbogen gemäss Satz 18.1 existiert, der den Fehler berücksichtigt, ist gegeben durch:

$$L(\varepsilon, \alpha) = \varepsilon \cdot \frac{P_{1,x}}{\sqrt{P_{1,x}} - P_{1,y}} = \varepsilon \cdot \frac{L + L \cdot \cos(\alpha)}{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}$$

- b) Bei der Vorgabe des Abstands L ergibt sich der folgende, maximale Fehler:

$$\varepsilon(L, \alpha) = L \cdot \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}} = L \cdot \frac{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}{L + L \cdot \cos(\alpha)}$$

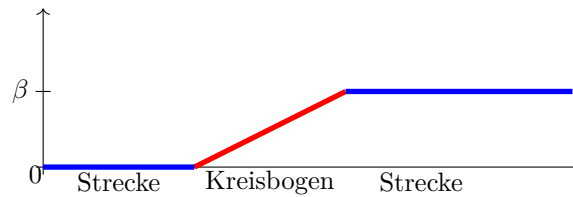


Abbildung 18.2.: Winkeländerung bei einer Verrundung mit einem Kreisbogen

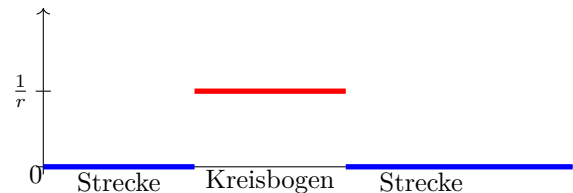


Abbildung 18.3.: Krümmungsverlauf bei der Verrundung mit einem Kreisbogen

Aus diesen Überlegungen ergeben sich Einschränkungen für den Einsatz dieser Strategie, die in der folgenden Bemerkung zusammengefasst sind.

Bemerkung. Folgende Bedingungen zum Anwenden der Strategie eines Kreises müssen erfüllt sein:

a)

$$P_0 \neq P_1$$

b)

$$\vec{t}_0 = -\vec{t}_1 \Leftrightarrow \alpha = 0$$

c)

$$\vec{t}_0 = \vec{t}_1 \Leftrightarrow \alpha = \pi$$

18.2. Bewertung

Die Verrundung mittels eines Kreisbogens führt zu einem stetigen Verlauf der Winkeländerung. Die Abbildung 18.2 stellt dies dar.

Durch die Verrundung mit einem Kreisbogen wird ist der Krümmungsverlauf der Kurve nicht stetig. Die Abbildung 18.3 zeigt, dass die Krümmung im Bereich der Strecken 0 ist und auf dem Kreisbogen konstant mit dem Wert $\frac{1}{r}$, wobei r der Radius des eingesetzten Kreisbogens ist. Aufgrund der Formel $a = \frac{v^2}{r}$ für eine Bewegung auf einem Kreisbogen weist der Beschleunigungsverlauf bei einer konstanten Bahngeschwindigkeit Stetigkeitssprüngen an den Übergängen aus.

In Abhängigkeit der Winkeländerung β an der Ecke S und der Toleranz ε kann die maximale Länge der Verkürzung der Strecken berechnet werden. Die Abbildung 18.4 zeigt das zugehörige Diagramm, wobei die Toleranz ε auf den Wert 1 gesetzt ist.

Der Bereich, der zur Verfügung gestellt wird, kann auch vorgegeben werden. In Abhängigkeit des Abstands L vom Eckpunkt S kann die Abweichung ε berechnet

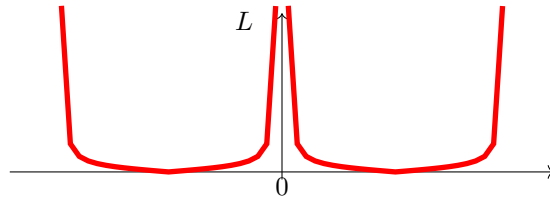


Abbildung 18.4.: Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$

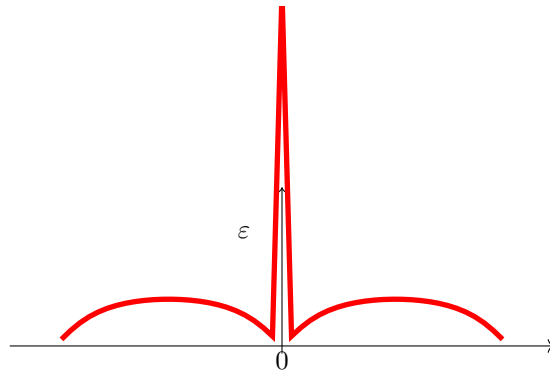


Abbildung 18.5.: Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen

werden. Die Abbildung 18.5 stellt die Funktion in Abhängigkeit von L und des Winkels α dar.

Die Abbildungen 18.4 und 18.5 zeigen die Kurven der Länge in Abhängigkeit der Winkeländerung bei einer festen Toleranz und den Fehler in Abhängigkeit der Winkeländerung bei vorgegebenen Länge. Falls die Winkeländerung minimal ist, dann ist der Fehler oder die Länge L extrem. In diesem Fall ist eine Verrundung mittels eines Kreisbogens nicht möglich. Um eine sinnvolle Strategie zu entwickeln, muss eine minimale Winkeländerung vorgegeben werden, ab der eine Verrundung mit einem Kreisbogen erlaubt ist. Ebenso ist eine maximale Winkeländerung zu definieren. Denn bei einer Winkeländerung von $\pm\pi$ handelt sich um eine Kehre. in diesem Fall ist eine Verrundung auch nicht möglich.

18.3. Beispiele

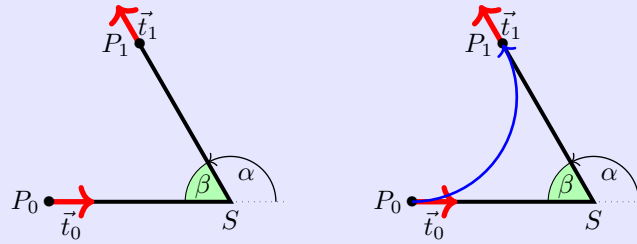
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$



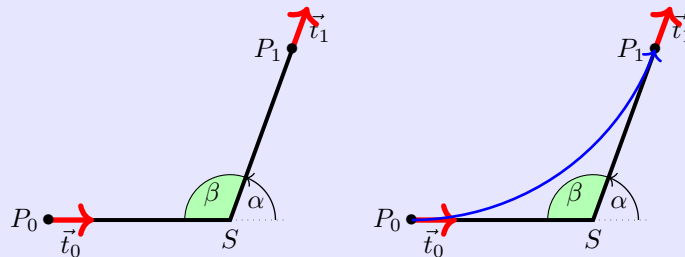
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$



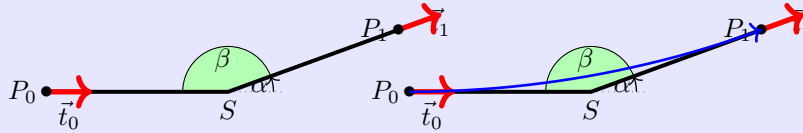
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

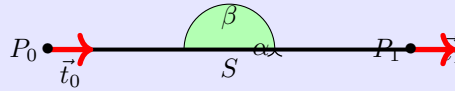


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = 0$ gegeben.

Hier existiert kein Verrundungskreisbogen.



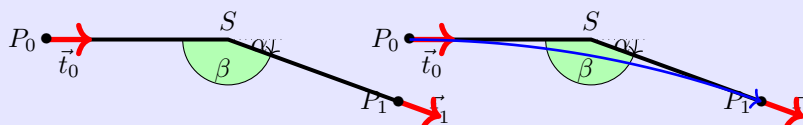
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{1}{9}\pi$$



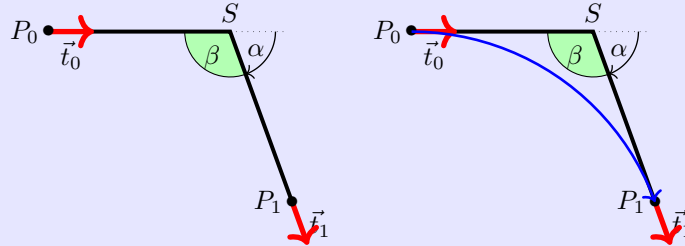
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$



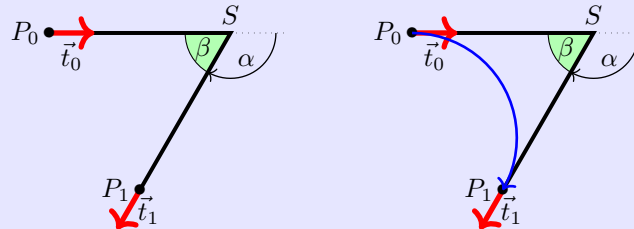
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



19. Maple-Dateien

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

19.1. Geometrien mit Maple

Die vorgestellten Algorithmen können gut mit Hilfe von Geometrien dargestellt werden. Dabei können zwei Aspekte untersucht werden. Einerseits ist der Aufwand für eine Umsetzung, die Rechengenauigkeit und die Stabilität eines Algorithmus interessant; andererseits sollen die Verfahren hinsichtlich ihres Nutzens bewertet und verglichen werden. Hierzu wurden Module für das Formelmanipulationssystem Maple [Wat17c] entwickelt.

Maple bietet die Umgebung, Algorithmen einfach und schnell zu implementieren. Darüber hinaus ist die grafische Darstellung mit einfachen Mitteln möglich. Allerdings ist eine einfache Arbeitsmappe von Maple nicht für sehr große Software-Projekte ausgelegt. Hier muss auf die Möglichkeit der Erstellung und Nutzung von Bibliotheken zurückgegriffen werden. Einerseits bietet Maple die Möglichkeit, eigene Bibliotheken, so genannte Module, zu erstellen. Auf diese Weise bleibt man innerhalb der Umgebung und der Syntax von Maple. Dieser Weg wird hier weiter verfolgt. Eine weitere Möglichkeit ist die Verwendung von Bibliotheken, die mittels einer höheren Programmiersprache, z.B. C++, erstellt wurde, den so genannten DLLs.

Im weiteren wird die Erstellung und Verwendung einer Testumgebung mit Hilfe von Module beschrieben. Zunächst werden die Geometrieelemente beschrieben, die in verschiedenen Module abgelegt sind, und deren Verwendung. Damit eigene Erweiterungen und Ergänzungen möglich sind, wird dann der Aufbau und die Verwendung eines Moduls in Maple erläutert.

19.2. Verwendete Geometrieelemente

Da es sich um eine Testumgebung für die vorgestellten Algorithmen, werden auch nur Geometrien verwendet, die beschrieben wurden.

- Punkte (**MPoint**)
- Strecken (**MLine**)
- Kreisbögen (**MArc**)
- Bézier-Kurven (**Bezier**)
- Polygonzüge (**MPolygon**)
- Geometrieliste (**MGeoList**)
- Hermite-Probleme (**MHermiteProblem**)
- Symmetrische Hermite-Probleme (**MHermiteProblemSym**)

Für jedes Geometrieelement wurde ein entsprechendes Modul angelegt, dessen Name in der obigen Liste angegeben ist. Die Auflistung, welche Funktionen zur Verfügung stehen, wird in einem weiteren Abschnitt dargestellt.

Bei der Umsetzung eines solchen Projekts wird schnell deutlich, dass bei einer Verwendung von mehreren Geometrieelementen eine klare Datenstruktur und ein wohldefinierter Zugriff auf die Daten unerlässlich ist. Zum Beispiel bei der Darstellung von Geraden ist die Entscheidung zu treffen, ob die Darstellung Punkt-Richtung oder mittels zweier Punkte erfolgen soll. Die Auswahl erfolgt im Allgemeinen in Abhängigkeit des Anwendungsfalls. Hier wird der Weg beschritten, dass aufgrund eines wohldefinierten Zugriffs auf die Daten die Verwendung beider Darstellungen möglich ist.

19.3. Aufbau der Datenstruktur

Die Idee ist, eine möglichst einheitliche und einfache Datenstruktur zu verwenden. Maple bietet zwar die Möglichkeit, Objekte zu definieren. Allerdings ist die Verwendung innerhalb einer prozeduralen Umgebung schwierig. Daher werden alle Daten grundsätzlich als Listen dargestellt. Das erste Listenelement enthält dabei stets eine Kennung des Elements `??`. Dann folgen die Daten, die wiederum Geometrieelemente sein können. Es ist zu beachten, dass der direkte Zugriff auf die Daten nicht unterbunden werden kann; hier ist der Anwender in der Pflicht.

Der Zugriff auf die Daten soll ausschließlich über Prozeduren eines Moduls erfolgen. Im folgenden ist beispielhaft die Datenstruktur für Punkte zu sehen:

```
[MVPOINT, [x, y]]
```

Die Erstellung eines Punkts erfolgt dann über eine Prozedur `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

Beim Aufruf in der Testdatei wird dann folgendes ausgegeben:

```
TestP0 := [PPoint", [10,15]]
```

Diese Datenstrukturen werden für die Berechnung der Geometrien verwendet. Mit ihnen wird in Funktionen bzw. Prozeduren gearbeitet. Anders als Variablen werden die Datenstrukturen und Prozeduren hier global und nicht lokal definiert. Unter dem Befehl `export` werden die Prozeduren zu Beginn des Moduls deklariert und sind so auch außerhalb des Moduls verwendbar. Soll beispielsweise eine Datenstruktur aus `MPoint` in einem anderen Modul oder außerhalb der Module verwendet werden, wird sie wie folgt aufgerufen:

```
P0 := MPoint:-New(x,y);
```

Zusätzlich zu den Modulen für die Geometrien gibt es ein Modul (`MConstant`) zum Speichern von Konstanten und Namen. Dort ist für `MVPOINT` z.B. „Point“ gespeichert oder z.B. eine Konstante für den Vergleich auf Null für reelle Zahlen.

Zuletzt gibt es die Testdatei, welche kein Modul ist, in der die Module geladen, aufgerufen und in ihrer Funktion getestet werden. Zu dieser Datei später im Punkt „1.4 Maple Testdatei“ mehr.

19.4. Struktur eines Moduls

In dem folgenden Abschnitt geht es um den Zweck von Modulen und deren Struktur eingegangen.

Es geht in dem Projekt darum die oben genannten Geometrien in einer Datenstruktur zu erfassen und sie in Maple darzustellen. Für die letztendliche Darstellung sind die Berechnungen und Formeln, die verwendet werden müssen, allerdings hinderlich. Module eignen sich sehr gut dafür die Berechnungen, die in Funktionen erfolgen,

zusammenzufassen und zu verstecken. So kann in Maple auf die wichtigen Funktionen zugegriffen werden, ohne dass man sieht, was in diesen steht.

Beispiel:

Die Funktion **Angle** aus dem Modul **MPoint**: Funktion zur Berechnung eines Winkels zwischen Ortsvektor und x-Achse:

```
Angle := proc(P)
  local alpha, x, y;
  x := GetX(P);
  y := GetY(P);
  alpha := 0;
  if abs(x) < 0.00001
  then
    alpha := 3*Pi/2;
  else
    alpha := Pi/2;
  end if;
else
  alpha := arctan(y/x);
  if x < 0
  then
    alpha := alpha + Pi;
  else
    if y < 0
    then
      alpha := alpha + 2*Pi;
    end if;
  end if;
end if;
return factor(alpha);
end proc;
```

Diese Funktion ist lang, stellt aber nur einen kleinen Teil des Programms für die Darstellung dar, um die es geht. Deshalb steht sie in dem Modul und kann so extern mit einem einzigen Befehl aufgerufen werden:

```
Winkel := MPoint:-Angle(P)
```

Im Folgenden Abschnitt wird die Struktur eines Moduls beschrieben. Da Maple hier sehr vielfältige Möglichkeiten bietet, findet eine Beschränkung statt. Es wird nur die Struktur der verwendeten Module, hier anhand des Moduls **MPoint**, beschrieben. Für tiefergehende Möglichkeiten wird hier auf das Handbuch von Maple [Wat17c] verwiesen.

Struktur:

Das Modul muss zunächst gestartet werden. Dies erfolgt mit dem Namen (hier immer ein groSSes M und die Geometrie) des Moduls und dem folgenden Befehl:

```
MPoint := module()
```

Danach müssen, ähnlich wie in Prozeduren, die Variablen und Funktionen definiert werden. Dabei kann man entweder lokal oder global deklarieren. Werden die Variablen

oder Prozeduren nur in dem Modul gebraucht und verändert, erfolgt die Deklaration mit dem Befehl `local` wie folgt:

```
local Variablenamen, mit, Komma, getrennt;
```

Sollen die Variablen oder Prozeduren auch auSSerhalb des Moduls verwendbar sein, wird mit `export` dies definiert:

```
export Funktionsnamen, mit, Komma, getrennt;
```

Daraufhin folgt die Angabe der Optionen:

```
option package;
```

die Beschreibung des Moduls:

```
description "Selbstgewählte Modulbeschreibung z.B. Modul für Punkte ";
```

und die Initialisierung des Moduls:

```
ModuleLoad := proc
  MVPPOINT:=MConstant:-GetPoint();
  print("Modul MPoint ist geladen");
end proc;
```

Ab hier wird programmiert wie sonst in Maple auch. Man verwendet die vorher definierten Prozedur- und Variablennamen und programmiert mit Befehlen die man auSSerhalb eines Moduls in Maple auch verwendet. Wichtig zu wissen ist, dass bei Modulen jede Funktion ständig abgerufen werden kann, die Reihenfolge der Funktionen also keine Rolle spielt.

Um zuletzt das Modul zu beenden, nutzt man den folgenden Befehl:

```
end module;
```

19.4.1. Genereller Aufbau eines Moduls

Zusammenfassend haben die Module also folgendes Gerüst:

```
Modulname := module()

  local Namen, mit, Komma, getrennt;

  export Namen, mit, Komma, getrennt;

  global Namen, mit, Komma, getrennt;1

  option package;

  description "Selbstgewählte Modulbeschreibung";

  ModuleLoad := proc()2
    MVPPOINT:=MConstant:-GetPoint();
    print("Modul MPoint ist geladen");
  end proc;

  Procedure1 := proc (Übergabeparameter)
```

¹möglich, aber in den Modulen nicht verwendet

²Beispiel `MPoint`


```
        inhalt;  
    end proc;  
  
    Procedure2 := proc (Übergabeparameter)  
        inhalt;  
    end proc;  
  
    ...  
  
end module;
```

19.4.2. Sicherung eines Moduls

Die Verwaltung von Module wird von Maple automatisch durchgeführt. Da aber die Module weitergegeben werden und einzeln zu bearbeiten sind, müssen einige Einstellungen vorgenommen werden. Daher wird bei jeder Maple-Datei des Projekts folgender Präfix verwendet:

```
1 restart;
2 with(LibraryTools);
3 lib := C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

Die 1. Zeile initialisiert das System. Die nächsten 3 Zeile ermöglichen das Arbeiten mit Archiven. In der zweiten Zeile werden die Werkzeuge geladen, so dass die Variable **lib** belegt werden kann. Alle Module werden in einem Archiv abgelegt; in diesem Beispiel ist es die Datei **Blending.mla** im Verzeichnis **C:/FH/Tools/Maple/MyLibs/**. Das Kommando **ThisModule := 'MArc';** enthält den Namen des aktuellen Moduls.

Ein Modul wird dann mittels des Befehls **savelib('ModuleName')** gespeichert. Es wird dann eine Datei **ModuleName.mla** angelegt. Je nach Konfiguration von Maple ist der eingestellte Pfad, wo die Datei automatisch angelegt wird, nicht beschreibbar. Dann kann man das System so konfigurieren, dass die mla-Datei im aktuellen Verzeichnis oder in einem Verzeichnis der eigenen Wahl abgelegt wird.

Falls der obige Vorspann für eine Maple-Datei verwendet wird, genügt zum Speichern des Moduls

```
savelib(ThisModule, lib);
```

19.4.3. Verwendung eines Moduls

Ein Modul, das abgespeichert wurde, kann nun in anderen Maple-Worksheets verwendet werden. Dazu wird der Befehl

```
with(ModuleName)
```

verwendet. Falls man einen speziellen Pfad verwendet hat, so kann Maple die Datei **ModuleName.mla** nicht finden. Dann muss der Pfad bekannt gemacht werden, z.B.:

```
savelibname := c:/Maple/MyLibs";", savelibname;
```

Nun kann man auf die Prozeduren des Moduls zugegriffen werden. Eine Übersicht über alle exportierten Prozeduren wird durch den Befehl

```
Describe(ModuleName)
```

angezeigt. Dabei wird eine List der Namen inklusiver der Namen der Übergabeparameter dargestellt. Falls eine Prozedur mit der Feld **description** ausgestattet ist, so wird dieser Text zusätzlich wiedergegeben.

19.4.4. Erstellung eines Maple-Moduls

Die Erstellung eines eigenen Moduls ist schnellst erledigt, falls man den obigen Rahmen verwendet. Allerdings sollte man vorher einige Überlegungen anstellen und einen Standard pflegen.

Bevor ein eigenes Modul erstellt wird, sollte das Datenmodell und die zugehörigen Prozeduren erarbeitet sein. Ein Programmablaufplan leistet hier gute Dienste. Die zentrale Aufgabe des Moduls ist in der Regel schnell ermittelt. Zusätzlich sollten aber folgende Regeln eingehalten werden.

Regel 1 Grundsätzlich erhält sowohl das Modul als auch jede Prozedur eine Beschreibung. Diese beschränkt sich nicht auf das optionale Argument **description**,

sonder wird jeder Prozedur vorangestellt. Die Beschreibung enthält grundsätzlich die Beschreibung der Aufgabe. Dabei werden auch die Voraussetzung genannt bzw. eine Fehlerbehandlung beschrieben. Dann folgt die Beschreibung aller Eingabeparameter, deren Funktion und deren Datenstruktur. Der Rückgabewert wird anschließend beschrieben.

Regel 2 Jedes Modul erhält eine Prozedur `Version()`, die die aktuelle Versionsnummer zurückliefert.

Regel 3 Daten sind nicht global. Um auf Daten zugreifen zu können, werden Prozeduren `Set*` und `Get*` zur Verfügung gestellt. Auch innerhalb der Prozeduren eines Moduls werden diese Prozeduren verwendet.

Regel 4 Für jede Prozedur wird mindestens eine Testfunktion geschrieben. Anhand der Testfunktion kann die Verwendung und ggfs. Besonderheiten dargestellt werden.

19.5. Funktionen der Module

Im folgendem werden die einzelnen Module aufgeführt. Die Datenstruktur jedes Moduls und alle Funktionen mit zugehöriger Aufgabe werden genannt.

19.5.1. Modul `MPoint`

`MPoint` ist ein Modul für Punkte und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur `New` für einen Punkt ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOINT, [x,y]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name `PPoint` und die Elemente sind die x- und die y-Koordinate für einen Punkt. Es wird hier kein Unterschied zwischen Punkt und Vektor gemacht. Ein Punkt kann auch als Vektor vom Koordinatenursprung zum Punkt gesehen werden.

Über die Get-Funktionen `GetX` und `GetY` kann man die Koordinaten des Punktes erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten/Vektoren dann gerechnet werden.

MPoint

E	New	Datenstruktur für einen Punkt
E	GetX	Lesen der x-Koordinate
E	GetY	Lesen der y-Koordinate
E	Angle	Berechnung des Winkels zwischen x-Achse und dem Punkt
E	Add	Errechnet eine Linearkombination zweier Punkte/Vektoren
E	Sub	Errechnet die Differenz zweier Punkte/Vektoren
E	Cos	Errechnet den Cosinus zwischen zwei Vektoren
E	Sin	Errechnet den Sinus zwischen zwei Vektoren
E	Scale	Skaliert einen Vektor mit einem Faktor
E	Perp	Errechnet einen Vektor, der orthogonal zum angegebenen Vektor ist
L	IllustrateXY	Plot Funktion zur Darstellung eines blauen Punktes
E	Illustrate	Darstellung eines blauen Punktes
E	Plot	Darstellung eines grünen Punktes
E	Plot2D	Darstellung eines Punktes mit eigenen Optionen
E	Length	Errechnet den Abstand des Punktes zum Koordinatenursprung
E	Uniform	Normiert den Vektor auf die Länge 1
E	LinetoVector	Errechnet Vektor aus einer Strecke
E	Distance	Errechnet den Abstand zwischen zwei Punkten

19.5.2. Modul MLine

MLine ist ein Modul für Strecken und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

[MVLINe, [P0,P1]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Line“ und die Elemente sind der Start- und der Endpunkt für eine Strecke. Die Datenstruktur **NewPointVector** ist ebenfalls eine Datenstruktur für eine Strecke, allerdings wird die Strecke hier über einen Startpunkt und einen Richtungsvektor definiert.

Über die Get-Funktionen **StartPoint** und **EndPoint** kann man Start- und Endpunkt der Strecke erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Strecken dann gerechnet werden.

MLine

E	New	Datenstruktur für eine Strecke (Zwei-Punkt-Form)
E	NewPointVector	Schaffung einer Strecke (Punkt-Richtungs-Form)
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	Position	Errechnen eines Punktes der auf der Strecke liegt
E	Plot2D	Darstellung eines Teils der Strecke ausgehend vom Startpunkt
E	Plot2DTangent	Darstellung eines Punktes mit Tangente
E	Plot2DTangentArrow	Darstellung eines Punktes mit Tangente (als Pfeil)
E	LineLine	Errechnung des Schnittpunktes zweier Geraden
E	AngleLineLine	Errechnung des Winkels zwischen zwei Geraden

19.5.3. Modul **MArc**

MArc ist ein Modul für Kreisbögen und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Kreisbogen ist eine Liste, die wie folgt aufgebaut ist:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **MArc** und die Elemente sind die x- und y-Koordinate für den Mittelpunkt, der Radius, der Startwinkel und die Winkeländerung für einen Kreisbogen.

Über die Get-Funktionen **GetM**, **GetMX**, **GetMY**, **GetR**, **GetPhi**, und **GetAlpha** kann man die Daten für einen Kreisbogen erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Kreisbögen dann gerechnet werden.

MArc

E	New	Datenstruktur für einen Kreisbogen
E	GetMX	Lesen der x-Koordinate des Mittelpunktes
E	GetMY	Lesen der y-Koordinate des Mittelpunktes
E	GetR	Lesen des Radius
E	GetPhi	Lesen des Startwinkels
E	GetAlpha	Lesen der Winkeländerung
E	GetM	Lesen des Mittelpunktes
E	Position	Errechnung eines Punktes auf dem Kreisbogen
E	Plot2D	Darstellung eines Teils des Kreisbogens ausgehend vom Startwinkel
E	Blend	Errechnung eines Kreisbogens aus einem symmetrischen Hermite-Problem

19.5.4. Modul **MBezier**

Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVBIEZIER, [PointList]]
```

Innerhalb des Moduls **MBezier** existieren die in folgender Tabelle aufgeführten Prozeduren.

MBezier

E	New	Manuelle Eingabe von Kontrollpunkten für eine Bézier-Kurve
E	Version	Ausgabe der Version
E	BlendCurvature	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem
E	BlendCurvatureEpsilon	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem mit vorgegebenen Fehler
E	Position	Position auf der Bézier-Kurve
E	GetTheta	Lesen des Winkels
E	GetEpsilon	Lesen der Toleranz
E	GetControlPoint	Lesen der Kontrollpunkte
E	Plot2D	Darstellung der Bézier-Kurve
E	PlotControlPoints	Darstellung aller Kontrollpunkte

19.5.5. Modul **MPolygon**

MPolygon ist ein Modul für Polygonzüge und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOLYGON, [PointList]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **PPolygon** und die Elemente sind beliebig viele Punkte.

Über die Get-Funktionen **GetPoint** und **GetN** kann man die Anzahl der Punkte und die Punkte selbst erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten bzw. dem Polygonzug dann gerechnet werden.

MPolygon

E	New	Datenstruktur für eine Punkteliste
E	GetPoint	Lesen des i-ten Punktes aus der Punkteliste
E	GetN	Bestimmung der Anzahl der Punkte in der Punkteliste
E	Length	Bestimmung der euklidischen Länge des Polygonzuges
E	Position	Berechnung eines Punktes auf dem Polygonzug
E	Tangents	Berechnung der Tangenten
L	Plot2DAll	Plotliste aller Punkte
E	Plot2D	Darstellung aller Punkte als Polygonzug
E	Plot2DTangent	Darstellung des Polygonzuges mit Tangenten
E	PlotPoints	Darstellung aller Punkte

19.5.6. Modul **MGeoList**

MGeoList ist ein Modul für eine Geometrieliste und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Geometrieliste ist eine Liste, die wie folgt aufgebaut ist:

```
[MVGEOLIST, []]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „GeoList“ und die Elemente sind beliebig viele einzelne Geometrien.

Über die Get-Funktionen **GeoGeo** und **GetN** kann man das i-te Element der Liste und die Anzahl der Elemente in der Liste erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Geometrien bzw. der Liste dann gerechnet werden. In den Funktionen **Length**, **Plot2DAll**, **Plot2D** und **Position** werden die Funktionen in sich selber aufgerufen. Dies ist möglich da die Funktionen unabhängig voneinander funktionieren.

MGeoList

E	New	Datenstruktur für eine Geometrieliste
E	Append	Anfügen eines Geometrieelements in die Datenstruktur
E	Prepend	Einfügen eines Geometrieelements als erstes Element der Liste
E	Replace	Ersetzen eines Geometrieelements durch ein Anderes
E	GetN	Bestimmung der Anzahl der Geometrieelemente in der Liste
E	GeoGeo	Lesen des i-ten Geometrieelements
E	Length	Errechnet die euklidische Länge der Geometrieelemente
E	Position	Berechnung eines Punktes auf der Geometrie
L	Plot2DAll	Plot Funktion für die Darstellung der Geometrieelemente
E	Plot2D	Darstellung eines Teils der Geometrieliste ausgehend vom ersten Element

19.5.7. Modul MHermiteProblem

MHermiteProblem ist ein Modul für Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „HermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten (Strecken).

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent** und **EndTangent** kann man die Punkte und ihre Tangenten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das Hermite-Problem dann gerechnet werden.

MHermiteProblem

E	New	Datenstruktur für ein Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	Plot2D	Darstellung des Hermite Problems

19.5.8. Modul MHermiteProblemSym

MHermiteProblemSym ist ein Modul für symmetrische Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein symmetrisches Hermite-Problem ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „SymHermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten, ihr Schnittpunkt, und der Abstand der Punkte zum Schnittpunkt.

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent**, **EndTangent**, **CrossPoint** und **ParameterL** kann man die Daten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das symmetrische Hermite-Problem dann gerechnet werden.

MHermiteProblemSym

E	New	Datenstruktur für ein Symmetrisches Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	ParameterL	Lesen des Abstandes
E	CrossPoint	Lesen des Schnittpunktes
E	Plot2D	Darstellung des symmetrischen Hermite Problems
E	Create	Erzeugung eines Sym. Hermite-Problems durch 3 Punkte
E	BlendArc	Verrundung des Eckpunktes durch einen Kreisbogen

19.5.9. Modul **MConstant**

MConstant ist ein Modul zum Speichern von festen Konstanten und Namen zur Kennung der Datenstrukturen. In der lokal deklarierten Funktionen, beginnend mit CV, werden die Konstanten zur Deklaration der verschiedenen Geometrien definiert. Dies sind Namen zur Erkennung der Geometrieelemente in der Testdatei. In den global deklarierten Funktionen, beginnend mit Get, erfolgt die Rückgabe der Namen zur Kennung der Geometrieelemente.

MConstant

L	NULLEPS	Konstante zum Vergleich auf Null
L	CVPOINT	Konstante für Geometrieelemente: Point
L	CVLINE	Konstante für Geometrieelemente: Line
L	CVARC	Konstante für Geometrieelemente: Arc
L	CVPOLYGON	Konstante für Geometrieelemente: Polygon
L	CVGEOLIST	Konstante für Geometrieelemente: GeoList
L	CVHERMITEPROBLEM	Konstante für Geometrieelemente: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Konst. für Geometrieelemente: SymHermiteProb
L	CVBIARC	Konst. für Geometrieelemente: Biarc
E	GetNullEps	Rückgabe des Nullvergleichs
E	GetPoint	Kennung für Punkte
E	GetLine	Kennung für Strecken
E	GetArc	Kennung für Kreisbögen
E	GetPolygon	Kennung für Polygone
E	GetGeoList	Kennung für Geometrielisten
E	GetHermiteProblemSymmetric	Kennung für symmetrische Hermite-Probleme
E	GetHermiteProblem	Kennung für Hermite-Probleme
E	GetBiarc	Kennung für Biarcs

19.5.10. Modul **MGeneralMath**

MGeneralMath ist ein Modul für allgemeine mathematische Funktionen und arbeitet mit den Datenstrukturen und Prozeduren.

MGeneralMath

E	MPoint	Datenstruktur für einen Punkt
E	MPointX	Lesen der x-Koordinate für einen Punkt
E	MPointY	Lesen der y-Koordinate für einen Punkt
L	MPointIllustrateXY	Plotstruktur für einen blauen Punkt
E	MPointIllustrate	Darstellung eines blauen Punktes
E	MPointPlot	Darstellung eines grünen Punktes
E	MLine	Datenstruktur für eine Strecke
E	MLineStartPoint	Lesen des Startpunktes für eine Strecke
E	MLineEndPoint	Lesen des Endpunktes für eine Strecke
E	MPointOnLine	Berechnung eines Punktes auf der Strecke
E	MLinePlot2D	Darstellung des Teils einer Strecke beginnend beim Startpunkt
E	MLineLine	Errechnen des Schnittpunktes zweier Strecken

19.5.11. Modul Biarc**Datenstruktur**

Voraussetzungen und Festlegungen:

Der Biarc soll ein dafür verwendet werden ein Hermite-Problem zu lösen. Ein Hermite Problem ist wie folgt definiert:

Gegeben seien zwei Punkte P_0 und P_1 mit zugehörigen normierten Tangenten $\vec{t}_0$ und $\vec{t}_1$. Der Biarc muss die Punkte so verbinden, dass die Tangenten des Start- und des Endpunktes des Biarcs mit den Tangenten des Hermites-Problems übereinstimmen.

Datenstruktur:

Die Datenstruktur **New** für einen Biarc muss zwei Kreisbögen enthalten.

Benötigt wird also die Datenstruktur für einen Kreisbogen: **MArc:-New**

Diese enthält wiederum die Koordinaten für den Mittelpunkt, den Radius, den Startwinkel und die Winkeländerung.

19.5.12. Modul MBiarc

MBiarc ist ein Modul für Biarcs und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Biarc ist eine Liste, die wie folgt aufgebaut ist:

[MVBIArc, [Arc0, Arc1]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Biarc“ und die Elemente sind zwei Kreisbögen.

Über die Get-Funktionen **GetArc0** und **GetArc1** kann man die beiden Kreisbögen für den Biarc erfassen. Mit den unten aufgeführten Funktionen kann man die Daten für den Biarc errechnen.

MBiarc

E	New	Datenstruktur für einen Biarc
E	GetArc0	Lesen des ersten Kreisbogens
E	GetArc1	Lesen des zweiten Kreisbogens
E	Circle	Berechnung des Kreises K_J aus den Daten des Hermite-Problems
E	Plot2DCircle	Darstellung des Kreises K_J
E	angle	Errechnet den Winkel vom Kreismittelpunkt zu Punkten auf dem Kreis
E	Plot2D	Darstellung des Biarcs
E	ConnectionPoint	Errechnung des Verbindungspunktes J (Equal Chord)
E	TangentTj	Errechnen der Tangente an J
E	Tangent Biarc	Drehung von Tj für den Biarc
E	BiarcCenter	Errechnen der Mittelpunkte der Kreisbögen des Biarcs
E	Biarc	Errechnen des Biarcs
E	Position	Ermitteln eines Punktes auf dem Biarc
E	Blend	Errechnen des Biarcs nur aus dem Hermite-Problem

19.6. Programmablaufplan**19.6.1. Gesamauf****19.6.2. Verrundung der Kurve**

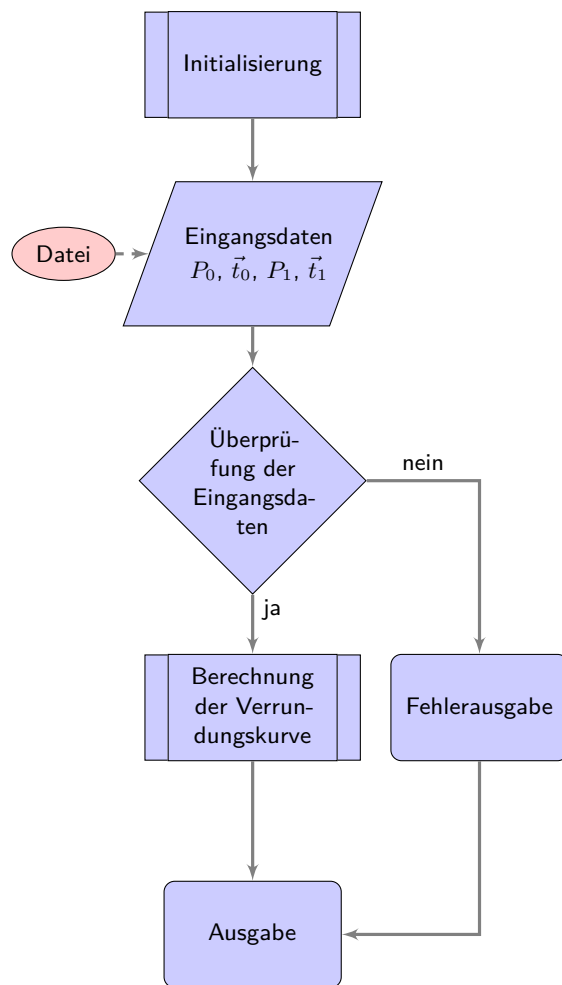


Abbildung 19.1.: Programmablaufplan „Eckenverrundung“

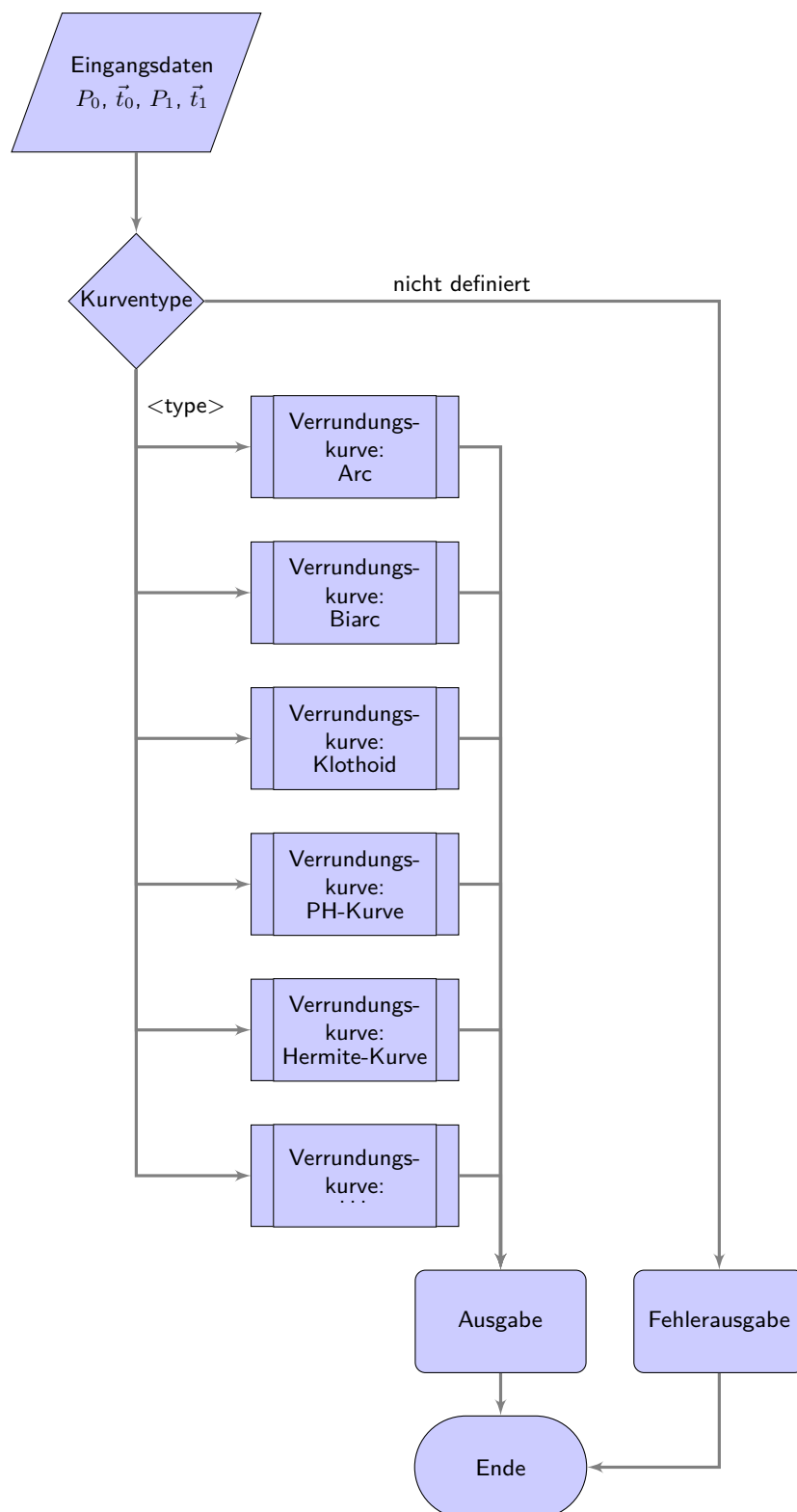


Abbildung 19.2.: Programmablaufplan „Auswahl der Verrundungsstrategien“

20. Representation of Python Programs

It is possible to integrate a part into the document, see 20.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 20.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```

1000 %%
1001 %% This is file 'rename-to-empty-base.tex',
1002 %% generated with the docstrip utility.
1003 %%
1004 %% The original source files were:
1005 %%
1006 %% fileerr.dtx (with options: 'return')
1007 %%
1008 %% This is a generated file.
1009 %%
1010 %% The source is maintained by the LaTeX Project team and bug
1011 %% reports for it can be opened at https://latex-project.org/bugs/
1012 %% (but please observe conditions on bug reports sent to that address!)
1013 %%
1014 %%
1015 %% Copyright (C) 1993–2025
1016 %% The LaTeX Project and any individual authors listed elsewhere
1017 %% in this file.
1018 %%
1019 %% This file was generated from file(s) of the Standard LaTeX ‘Tools
1020 %% Bundle’.
1021 %%
1022 %%
1023 %% It may be distributed and/or modified under the
1024 %% conditions of the LaTeX Project Public License, either version 1.3c
1025 %% of this license or (at your option) any later version.
1026 %% The latest version of this license is in
1027 %% https://www.latex-project.org/lppl.txt
1028 %% and version 1.3c or later is part of all distributions of LaTeX
1029 %% version 2005/12/01 or later.
1030 %%
1031 %% This file may only be distributed together with a copy of the LaTeX
1032 %% ‘Tools Bundle’. You may however distribute the LaTeX ‘Tools Bundle’
1033 %% without such generated files.
1034 %%
1035 %% The list of all files belonging to the LaTeX ‘Tools Bundle’ is
1036 %% given in the file ‘manifest.txt’.
1037 %%
1038 \message{File ignored}
1039 \endinput
1040 %%
1041 %% End of file 'rename-to-empty-base.tex'.

```

../Code/HelloWorld/Blink.py

21. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 21.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 21.1.: The program “Hello World” for an Arduino microcontroller board is inserted from the file `Blink.ino`.

```

1000 /**
1001  *
1002  *   @file Blink.ino
1003  *
1004  *   @brief Simple program for testing the configuration
1005  *
1006  *   Turns an LED on for one second, then off for one second, repeatedly.
1007  *
1008  *   Most Arduinos have an on-board LED you can control. On the UNO, MEGA
1009  *   and ZERO
1010  *   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
1011  *   set to
1012  *   the correct LED pin independent of which board is used.
1013  *   If you want to know what pin the on-board LED is connected to on your
1014  *   Arduino
1015  *   model, check the Technical Specs of your board at:
1016  *
1017  *   https://www.arduino.cc/en/Main/Products
1018  *
1019  *   modified 8 May 2014
1020  *   by Scott Fitzgerald
1021  *   modified 2 Sep 2016
1022  *   by Arturo Guadalupi
1023  *   modified 8 Sep 2016
1024  *   by Colby Newman
1025  *
1026  *   This example code is in the public domain.
1027  *
1028  *   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1029  */
1030
1031
1032 /**
1033  *   @brief the setup function runs once when you press reset or power the
1034  *   board
1035  *
1036  *   standard function of Arduino sketches
1037  *
1038  *   Initialization of the pin LED_BUILTIN as output
1039  *
1040  *   @param —
1041  *
1042  *   @return void
1043  */
1044 void setup() {
1045     // initialize digital pin LED_BUILTIN as an output.
1046     pinMode(LED_BUILTIN, OUTPUT);
1047 }
1048
1049 /**
1050  *   @brief the loop function runs over and over again forever
1051  *
1052  *   standard function of Arduino sketches
1053  *
1054  *   swichting the led on / off for 1sec.
1055  *
1056  *   @param —
1057  *
1058  *   @return void
1059  */
1060 void loop() {
1061     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the
1062     voltage level)
1063     delay(1000); // wait for a second
1064     digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the
1065     voltage LOW
1066     delay(1000); // wait for a second
1067 }

```

../Code/Arduino/Blink/Blink.ino

22. Erstes Kapitel

...

Eine Speicherprogrammierbare Steuerung (SPS) ist ...

Eine Computerized Numerical Control (CNC) benötigt eine SPS (SPS) zur ...

23. CAGD

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Das Buch CAGD von Gerald Farin ist ein Klassiker über Splines. [Far02]

Die Norm 66025 zur Programmierung von CNC-Maschinen ist ebenfalls ein Klassiker; sie behandelt allerdings keine Splines. [DIN66025-2]

Herr F. Farouki hat sich sowohl mit der Programmierung von CNC-Maschinen als auch mit Splines beschäftigt. Sein Artikel¹ über einen Echtzeitinterpolator zeigt dies auch. [FS17]

Ein neue Dimension der Werkzeugmaschinen sind durch die Erfindung von 3D-Drucker entstanden. [Rus+07]

Ein anderer Aspekt der Automatisierungstechnik ist die Kommunikation. Durch die 5G-Technik ist hier ein weiterer Meilenstein erreicht worden.²

¹Mitautor ist J. Srinathu

²Zaf+20.

A. Zeichnungen mit tikz

Das Paket tikz ist ein mächtiges Werkzeug zu erstellen von Grafiken. Es existieren vielen Einführungen. Hier werden nur die ersten Schritte gezeigt, so dass man leicht Flussdiagramme erzeugen kann.

Zeichnen einer Linie und Pfeile

B. Kriterien für eine gutes L^AT_EX-Projekt

Ein guter Bericht zeichnet sich nicht nur durch den guten Inhalt aus, sondern erfüllt auch formale Aspekte. Die folgende Liste soll helfen, grundlegende Regeln einzuhalten. Vor der Abgabe sollten alle Punkte geprüft und abgehakt werden.

- ☐ Wird eine geeignete Verzeichnisstruktur verwendet?
- ☐ Wird eine geeignete Aufteilung in Dateien verwendet?
- ☐ Werden aussagekräftige Verzeichnis- und Dateinamen verwendet?
 - ☐ Werden Verzeichnis- und Dateinamen wie z.B. report oder Hausarbeit vermieden?
 - ☐ Werden für Bilder aussagekräftige Namen und nicht „image1“ verwendet?
 - ☐ Sind die Verzeichnis- und Dateinamen nicht zu lang?
 - ☐ Werden Sonderzeichen und Freizeichen vermieden?
- ☐ Werden Befehle wie `\newline` und `\\` vermieden?
- ☐ Werden die L^AT_EX-Dateien übersichtlich gestaltet?
 - ☐ Werden in den L^AT_EX-Dateien Einrückungen verwendet?
 - ☐ Werden Freizeilen eingefügt?
 - ☐ Wird im Header der Dateien das Projekt erwähnt?
 - ☐ Wird im Header der Dateien die Hauptquellen erwähnt?
 - ☐ Wird im Header der Dateien der Autor erwähnt?
- ☐ Werden alle notwendigen Dateien abgegeben?
- ☐ Werden die temporären Dateien gelöscht?
- ☐ Werden Grafiken selber erstellt?
- ☐ Werden die Quellen der Bilder angegeben?
- ☐ Wird mit dem Kommando `\cite` zitiert?
- ☐ Wird eine bib-Datei verwendet?
- ☐ Sind die Einträge der bib-Datei übersichtlich?
- ☐ Sind die Einträge der bib-Datei so editiert, dass sie korrekt dargestellt werden?
- ☐ Werden die korrekten Typen für die bib-Einträge verwendet?
- ☐ Werden aussagekräftige Schlüssel in den bib-Dateien verwendet?

Leider kann die Liste nicht vollständig sein, aber sie liefert einige Anhaltspunkte.

Index

3D-Drucker, 123

5G, 123

Speicherprogrammierbare Steuerung, 121

Datei

 Blink.ino, 120

 Blink.py, 118

 Mag_raw.txt, 35

 PackageExample.py, 85

 requirements.txt, 83

Inertialmesseinheit

siehe IMU, 11, 32

Acoustic Overload Point

siehe AOP, 11, 37

AOP, 11, 37

CAGD, 123

 Splines, 123

CNN, 11, 121

Computerized Numerical Control

siehe CNN, 11, 121

Example, 85

 Description, 85

 Installation, 85

 Introduction, 85

IMU, 11, 32, 34, 35

PDM, 11, 37

Pulse Density Modulation

siehe PDM, 11, 37

Speicherprogrammierbare Steuerung

siehe SPS, 11, 121

SPS, 11, 121, *siehe* Speicherprogrammierbare Steuerung